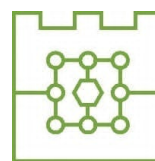




**Politechnika Krakowska
im. Tadeusza Kościuszki**
Wydział Informatyki i Telekomunikacji



Michał Bąk

Numer albumu: 144753

**Porównanie metod uczenia przez wzmacnianie w Ultimate Texas
Hold'em z niepełną informacją: wpływ reprezentacji stanu i funkcji
nagrody na jakość strategii**

**Comparison of reinforcement learning methods in Ultimate Texas Hold'em under
imperfect information: the impact of state representation and reward function on
strategy quality**

**Praca dyplomowa magisterska
na kierunku Informatyka**

Praca wykonana pod kierunkiem:
mgr inż. Piotr Biskup

Kraków, 2026

SPIS TREŚCI

1	Wstęp	5
1.1	Cel pracy	5
1.2	Zakres pracy	5
1.3	Problem badawczy	6
1.4	Struktura pracy	6
2	Podstawy teoretyczne	9
2.1	Poker jako gra z niepełną informacją	9
2.2	Texas Hold'em	9
2.3	Ranking układów pokerowych	9
2.4	Ultimate Texas Hold'em	10
2.4.1	Charakterystyka gry	10
2.4.2	Przebieg rozdania	10
2.4.3	Zakłady i decyzje gracza	11
2.4.4	Rozstrzyganie rozdania	11
2.4.5	Zakłady poboczne	12
2.5	Wartość oczekiwana i przewaga kasyna	12
2.6	Strategie bazowe i ocena jakości strategii	13
2.7	Uczenie przez wzmacnianie	13
2.7.1	Podstawowe pojęcia	13
2.8	Modelowanie UTH jako procesu decyzyjnego	14
3	Projekt i implementacja środowiska Ultimate Texas Hold'em	15
3.1	Założenia projektowe środowiska	15
3.2	Architektura rozwiązania	16
3.3	Model kart i sterowanie talią	18
3.3.1	Reprezentacja karty	18
3.3.2	Standardowa talia	18
3.3.3	Abstrakcja źródła kart i talia testowa	19
3.4	Ocena układów pokerowych	19
3.4.1	Reprezentacja wartości ręki	19
3.4.2	Ocena układu pięciokartowego	20

3.4.3	Wybór najlepszej ręki	21
3.5	Stan wewnętrzny i obserwacja agenta	21
3.5.1	Pełny stan rozdania	21
3.5.2	Obserwacja agenta	23
3.5.3	Projekcja stanu na obserwację	23
3.6	Maszyna stanów i przestrzeń akcji	24
3.7	Showdown i rozliczanie zakładów	25
3.7.1	Przebieg showdownu	25
3.7.2	Kwalifikacja krupiera	26
3.7.3	Model rozliczenia zakładów	27
3.7.4	Rozliczenie zakładu Blind	27
3.7.5	Rozliczenie Ante, Play i spasowania	28
3.8	Funkcja nagrody i wynik epizodu	28
3.9	Interfejs silnika	29
3.9.1	Tworzenie silnika i rozpoczęcie epizodu	29
3.9.2	Wykonanie decyzji	29
3.9.3	Właściwości publiczne	30
3.10	Rejestrowanie przebiegu i walidacja środowiska	30
3.10.1	Rejestrowanie historii i jej rekordy	31
3.10.2	Poziomy testowania	31
3.11	Ograniczenia środowiska	32
3.12	Podsumowanie	33
4	Agenci bazowi i infrastruktura eksperymentalna	35
4.1	Wspólny interfejs agenta	35
4.2	Prowadzenie pojedynczego epizodu	35
4.3	Agent losowy	35
4.4	Agent regułowy	36
4.4.1	Reguły decyzyjne	36
4.4.2	Implementacja strategii regułowej	37
4.5	Symulacja wielu epizodów i jej powtarzalność	37
4.6	Metryki oceny	38
4.7	Porównanie agentów bazowych	38
4.8	Zapis wyników eksperymentu	39
4.9	Walidacja infrastruktury eksperymentalnej	39
4.10	Podsumowanie	40
5	Metody uczenia przez wzmacnianie	41
5.1	Reprezentacja stanu	41
5.1.1	State A: reprezentacja surowa	41

5.1.2	State B: reprezentacja z cechami domenowymi	42
5.2	Funkcja nagrody	42
5.3	Deep Q-Network	43
5.3.1	Sieć Q i maskowanie akcji	44
5.3.2	Eksploracja epsilon-greedy	44
5.3.3	Bufor doświadczeń	45
5.3.4	Sieć docelowa i cel Bellmana	45
5.3.5	Strata i optymalizacja	46
5.4	Policy Gradient – REINFORCE	47
5.4.1	Sieć polityki i wybór akcji	47
5.4.2	Uczenie metodą REINFORCE	48
5.4.3	Reprodukowalność i diagnostyka	49
5.5	Porównanie podejść	49
6	Metodyka eksperymentów	51
6.1	Macierz eksperymentalna	51
6.2	Powtarzalność eksperymentów	52
6.3	Budżet treningowy	52
6.4	Walidacja okresowa	52
6.5	Rozszerzona analiza treningu	53
6.6	Ewaluacja końcowa	53
6.7	Porównania sparowane	54
7	Wyniki eksperymentów	55
7.1	Wyniki końcowej ewaluacji	55
7.2	Wpływ reprezentacji stanu	57
7.3	Wpływ funkcji nagrody	58
7.4	Przebieg procesu uczenia	60
7.5	Wpływ zwiększenia budżetu treningowego	62
7.6	Stabilność procesu uczenia	64
7.7	Analiza zachowania wyuczonych polityk	66
7.8	Dyskusja wyników	69

1. Wstęp

Od zawsze interesowały mnie gry karciane jako obszar zastosowania metod sztucznej inteligencji, ponieważ łączą w sobie elementy losowości oraz podejmowania decyzji przy niepełnej informacji. W klasycznych problemach uczenia maszynowego model uczy się na podstawie gotowego zbioru danych, a w grach agent musi samodzielnie podejmować decyzje, obserwować ich konsekwencje i poprawiać swoją strategię.

Szczególnie interesującym mnie przypadkiem są gry pokerowe, w których wynik pojedynczego rozdania nie zależy jedynie od posiadanych kart, ale również od decyzji podjętych w ramach kolejnych etapów gry. Chciałbym zwrócić uwagę na Ultimate Texas Hold'em, kasynową odmianę pokera bazującą na zasadach Texas Hold'em, w której gracz rywalizuje nie z innymi graczami, a z krupierem. Odmiana ta ma ograniczoną liczbę możliwych akcji, jasno określone reguły wypłat oraz sekwencyjny przebieg rozdania. Te cechy, w moim mniemaniu, czynią ją odpowiednim problemem do rozważenia w ramach uczenia przez wzmacnianie.

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) pozwala analizować sytuacje, w których agent uczy się strategii poprzez interakcję ze środowiskiem. W przypadku Ultimate Texas Hold'em środowiskiem może być symulator gry, obserwacją informacje dostępne graczowi w bieżącym etapie rozdania, akcją decyzja gracza, natomiast nagrodą końcowy wynik finansowy rozdania. Takie podejście umożliwia mi badanie, w jaki sposób różne reprezentacje stanu oraz różne definicje funkcji nagrody wpływają na jakość wyuczonej strategii.

1.1. Cel pracy

Głównym celem tej pracy jest ocena przydatności wybranych metod uczenia przez wzmacnianie do podejmowania decyzji w Ultimate Texas Hold'em. Analizuję również wpływ sposobu reprezentacji obserwacji oraz konstrukcji funkcji nagrody na przebieg uczenia i jakość otrzymywanej strategii.

Za cel praktyczny obrałem zaprojektowanie i implementację kompletnego procesu badawczego obejmującego środowisko gry, agentów bazowych i uczących się, mechanizm prowadzenia treningu oraz powtarzalną ocenę uzyskanych strategii. Porównanie zawiera zarówno różnice pomiędzy metodami uczenia, jak i ich odniesienie do strategii bazowych, w tym agenta losowego i agenta regułowego.

1.2. Zakres pracy

Zakres pracy obejmuje zaprojektowanie i implementację środowiska odwzorowującego podstawowy wariant Ultimate Texas Hold'em, w którym uwzględniłem podstawowe elementy rozgrywki: karty gracza i krupiera, karty wspólne, obowiązkowe zakłady Ante

i Blind oraz decyzyjny zakład Play. W mojej wersji środowiska pominąłem zakłady poboczne, takie jak Trips oraz Progressive, ponieważ nie są one bezpośrednio związane z głównym procesem decyzyjnym agenta.

W ramach pracy przygotowałem różne warianty numerycznej reprezentacji obserwacji agenta, utworzonej na podstawie dostępnej części stanu gry. Reprezentacja obserwacji obejmuje między innymi informacje o kartach gracza, kartach wspólnych, aktualnym etapie rozdania oraz dostępnych akcjach. Podałem analizie również wpływ sposobu przekształcania finansowego wyniku rozdania w sygnał nagrody.

Założenia niniejszej pracy są następujące:

1. Implementacja środowiska odwzorowującego podstawowy wariant UTH.
2. Implementacja mechanizmu oceny układów, przebiegu rozdania i rozliczania zakładów.
3. Przygotowanie agentów bazowych (ang. *baseline agents*) stanowiących punkty odniesienia dla metod uczących się.
4. Implementacja wybranych metod uczenia przez wzmacnianie.
5. Przygotowanie i porównanie różnych reprezentacji obserwacji.
6. Przygotowanie i porównanie wybranych funkcji nagrody.
7. Opracowanie powtarzalnej procedury treningu i oceny agentów.
8. Analiza jakości strategii, przebiegu uczenia oraz wyników uzyskanych względem pozostałych metod i strategii bazowych.

1.3. Problem badawczy

Problem badawczy, który postanowiłem rozważyć, dotyczy wpływu wyboru metody uczenia przez wzmacnianie, reprezentacji obserwacji oraz funkcji nagrody na przebieg uczenia oraz jakość strategii uzyskiwanej w Ultimate Texas Hold'em.

Analizie poddałem także to, które konfiguracje pozwalają osiągnąć najwyższy średni wynik netto i najmniejszą oczekiwaną stratę, jak stabilny jest proces uczenia oraz czy uzyskane strategie przewyższają przyjęte strategie bazowe. Agenci bazowi służą jako punkty odniesienia do oceny metod uczenia przez wzmacnianie.

1.4. Struktura pracy

Struktura pracy składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiam cel, zakres oraz problem badawczy pracy. W rozdziale drugim zawieram podstawy teoretyczne dotyczące pokera, Texas Hold'em, Ultimate Texas Hold'em, wartości oczekiwanej, przewagi kasyna oraz uczenia przez wzmacnianie.

W kolejnych rozdziałach opisuję projekt środowiska symulacyjnego, sposób reprezentacji stanu gry, funkcję nagrody oraz zastosowane metody uczenia przez wzmacnianie. Następnie przedstawiam eksperymenty, uzyskane wyniki oraz analizę porównawczą agentów. W ostatnim rozdziale zawieram podsumowanie pracy, wnioski oraz możliwe kierunki dalszego rozwoju.

2. Podstawy teoretyczne

2.1. Poker jako gra z niepełną informacją

Poker jest zbiorem gier karcianych, w których wynik zależy zarówno od losowego rozdania kart, jak i od decyzji podejmowanych przez graczy. W przeciwieństwie do gier z pełną informacją (np. szachy) uczestnicy rozgrywki nie znają wszystkich elementów stanu gry. Ukryte pozostają przede wszystkim karty przeciwników, co sprawia, że decyzje podejmowane są w warunkach niepełnej informacji.

Właśnie z uwagi na tę niepełną informację uznałem pokera za ciekawy problem badawczy. W pokerze pojedyncza decyzja gracza nie powinna być oceniana wyłącznie na podstawie uzyskanego wyniku na koniec rozdania. Bardziej istotne jest, jak dana decyzja wpływa na wartość oczekiwaną (ang. *expected value*, EV) w dłuższym horyzoncie czasowym, przy wielu rozdaniach.

2.2. Texas Hold'em

Odmiana pokera Texas Hold'em jest jedną z najpopularniejszych na świecie. W tej odmianie każdy z graczy otrzymuje po dwie zakryte karty własne (ang. *hole cards*), znane tylko temu graczowi, natomiast na stole etapami pojawia się ostatecznie pięć kart wspólnych (ang. *community cards*). Gracz, wykorzystując karty wspólne oraz swoje karty własne, musi utworzyć możliwie najlepszy układ pięciokartowy.

Sama rozgrywka składa się z kilku etapów: etapu przed flopem (ang. *preflop*), flopa (ang. *flop*), czyli odkrycia trzech pierwszych kart wspólnych, turna (ang. *turn*), czyli czwartej karty wspólnej, rivera (ang. *river*), czyli odkrycia piątej i ostatniej karty wspólnej, oraz showdownu (ang. *showdown*), czyli końcowego porównania układów. Warto dodać, że pomiędzy odkrywaniem kolejnych kart wspólnych dokonuje się palenia kart (ang. *burning*), czyli odkładania wierzchniej karty z talii na bok. Każdy z tych etapów poprzedza runda licytacji, w której gracze mogą stawiać zakłady, podbijać je lub spasować.

2.3. Ranking układów pokerowych

Zarówno w Texas Hold'em, jak i w Ultimate Texas Hold'em końcowy wynik rozdania zależy od siły najlepszego układu pięciokartowego utworzonego z dostępnych kart (pięciu kart wspólnych oraz dwóch własnych). Układy pokerowe mają ustaloną hierarchię, która pozwala jednoznacznie porównać ręce graczy lub krupiera w odmianie UTH. Przykładowy ranking układów od najsilniejszego do naj słabszego przedstawiam w tabeli 2.1 [1].

Najsilniejszym układem jest poker królewski, natomiast naj słabszym układem jest wysoka karta. Jeśli gracz i krupier posiadają układ tej samej kategorii (np. obaj mają parę), o zwycięstwie decydują wartości kart składających się na dany układ oraz karty

dodatkowe (ang. *kickers*).

Tabela 2.1. Ranking układów pokerowych od najsilniejszego do najslabszego

Układ	Opis
Poker królewski	A, K, Q, J, 10 w tym samym kolorze
Poker	Pięć kolejnych kart w tym samym kolorze
Kareta	Cztery karty tej samej wartości
Full	Trójka oraz para
Kolor	Pięć kart w tym samym kolorze
Strit	Pięć kolejnych kart
Trójka	Trzy karty tej samej wartości
Dwie pary	Dwa różne układy par
Para	Dwie karty tej samej wartości
Wysoka karta	Brak silniejszego układu

2.4. Ultimate Texas Hold'em

2.4.1. Charakterystyka gry

Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em. Różnica polega na tym, że gracz rywalizuje z krupierem, który reprezentuje kasyno, a nie z innymi graczami, co oznacza, że konieczność blefowania (ang. *bluffing*) jest tu zbędna. Istotną cechą Ultimate Texas Hold'em jest również ograniczona liczba momentów decyzyjnych: gracz może wykonać zakład Play (ang. *Play bet*) tylko raz w trakcie rozdania, a im wcześniej podejmie tę decyzję, tym wyższy zakład może postawić, kosztem mniejszej liczby dostępnych informacji [1], [2]. Szczegółowe wysokości zakładu Play na poszczególnych etapach przedstawia tabela 2.2.

2.4.2. Przebieg rozdania

Rozdanie rozpoczyna się od postawienia przez gracza obowiązkowych zakładów Ante (ang. *Ante bet*) oraz Blind (ang. *Blind bet*). Oba te zakłady stawia się w tej samej wysokości. Opcjonalnie gracz może również postawić zakład poboczny (ang. *side bet*), na przykład Trips, który opisuję szerzej w dalszej części tego rozdziału [1].

Kiedy zakłady Blind oraz Ante zostaną postawione, krupier rozdaje graczowi i sobie po dwie karty własne. Karty krupiera pozostają przy tym dla gracza zakryte. Teraz, w fazie preflop, po ujrzeniu swoich dwóch kart, gracz stoi przed pierwszą decyzją: może zaczekać (ang. *check*) lub wykonać zakład Play. Czekanie powoduje przejście do kolejnego etapu gry, jakim jest flop: następuje wtedy palenie jednej karty z talii i odkrycie kolejnych trzech kart wspólnych, a gracz znów podejmuje decyzję, czy chce zaczekać, czy wykonać zakład Play. Jeżeli gracz czeka również po flopie, następuje palenie karty i odkrywane są dwie ostatnie karty wspólne (turn i river), a gracz musi wybrać pomiędzy spasowaniem a wykonaniem zakładu Play. Wysokości zakładu Play na każdym z tych etapów przedstawiono w tabeli 2.2 [1], [2].

2.4.3. Zakłady i decyzje gracza

Zakłady Blind i Ante są obowiązkowe, natomiast zakład Play jest opcjonalny, zależny od decyzji gracza i możliwy do postawienia raz w trakcie rozdania. To właśnie zakład Play, z punktu widzenia modelowania problemu jako środowiska uczenia przez wzmocnianie, jest najważniejszy, ponieważ jego wysokość i moment wykonania wynikają z decyzji agenta. Ante i Blind traktuję jako początkowy koszt udziału w rozdaniu, natomiast zakład Play odzwierciedla wybór strategii w aktualnym stanie gry.

Tabela 2.2. Dostępne decyzje gracza w Ultimate Texas Hold'em

Etap	Dostępne decyzje	Wysokość zakładu Play
Preflop	czekanie lub zakład Play	3x albo 4x Ante
Flop	czekanie lub zakład Play	2x Ante
Po odkryciu całego stołu	spasowanie lub zakład Play	1x Ante

2.4.4. Rozstrzygnięcie rozdania

Po dokonaniu decyzji przez gracza o wykonaniu zakładu Play krupier odstawia swoje dwie karty własne. Gracz oraz krupier tworzą najlepszy możliwy układ pięciokartowy z siedmiu dostępnych kart. Następnie układy są porównywane zgodnie z hierarchią układów pokerowych.

Krupier kwalifikuje się do gry, jeżeli posiada co najmniej parę. Jeżeli krupier się nie kwalifikuje, zakład Ante jest zwracany graczowi. Zakład Play jest rozliczany na podstawie bezpośredniego porównania układu gracza i krupiera. W przypadku zwycięstwa gracza zakład Play wypłacany jest w stosunku 1:1, w przypadku przegranej gracz traci zakład Play, a w przypadku remisu zakład jest zwracany [1], [2].

Zakład Blind ma odrębną logikę rozliczania i nie jest wypłacany za każdy zwycięski układ gracza. Jeżeli gracz pokona krupiera układem o wartości co najmniej strita, Blind wypłacany jest zgodnie z tabelą wypłat przedstawioną w tabeli 2.3. Jeżeli gracz wygra rozdanie układem słabszym niż strit, zakład Blind jest zwracany. W przypadku przegranej gracza zakład Blind jest tracony, a w przypadku remisu zwracany [1].

Tabela 2.3. Tabela wypłat zakładu Blind przyjęta w modelu środowiska

Układ gracza	Wypłata
Poker królewski	500:1
Poker	50:1
Kareta	10:1
Full	3:1
Kolor	3:2
Strit	1:1
Pozostałe układy	Zwrot

2.4.5. Zakłady poboczne

W wielu wariantach Ultimate Texas Hold'em dostępne są również zakłady poboczne, takie jak Trips i Progressive. Zakład Trips jest opcjonalny i wypłacany na podstawie końcowego układu gracza, niezależnie od tego, czy gracz pokonał krupiera. Najczęściej wygrywa wtedy, gdy końcowy układ gracza ma wartość co najmniej trójki. Zakłady poboczne mogą różnić się w zależności od kasyna i stosowanej tabeli wypłat [1].

Głównym celem pracy jest modelowanie sekwencyjnego procesu decyzyjnego dotyczącego zakładu Play. Ponieważ zakłady poboczne są opcjonalne, a wysokość ich wypłat zależy od konkretnego kasyna, zdecydowałem się ich nie uwzględniać w modelu środowiska.

2.5. Wartość oczekiwana i przewaga kasyna

Wartość oczekiwana (ang. *expected value*, EV) określa średni wynik finansowy decyzji lub strategii przy założeniu dużej liczby powtórzeń gry. W kontekście Ultimate Texas Hold'em pojedyncze rozdanie może zakończyć się zarówno zyskiem, jak i stratą, jednak skuteczność strategii należy oceniać na podstawie średniego wyniku uzyskiwanego w wielu rozdaniach.

Przewaga kasyna (ang. *house edge*) jest matematyczną miarą określającą oczekiwany zysk kasyna względem zakładu gracza. Bynajmniej nie oznacza ona, że kasyno wygrywa każde pojedyncze rozdanie, lecz że przy dużej liczbie rozegranych rozdań średni wynik będzie przesunął się na korzyść kasyna. Oczywiście, gracz może osiągać krótkoterminowe zyski, jednak w długim okresie wynik gry powinien zbliżać się do wartości oczekiwanej wynikającej z zasad gry [3].

W Ultimate Texas Hold'em przewaga kasyna podawana jest względem początkowego zakładu Ante. Metryki, które przyjąłem, opierają się na wyniku netto względem Ante. Nawet niewielka procentowa przewaga kasyna prowadzi do istotnej oczekiwanej straty w długim horyzoncie czasowym.

W tabeli 2.4 przedstawiam przykładowe wartości przewagi kasyna dla wybranych gier kasynowych, w tym Ultimate Texas Hold'em [4].

Tabela 2.4. Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych

Gra	Przewaga kasyna
Blackjack	0,28%
Ultimate Texas Hold'em	2,19%
Ruletka europejska	2,70%
Ruletka amerykańska	5,26%
Automaty do gier	2–15%

Tabela 2.4 pokazuje, że przewaga kasyna jest cechą konstrukcyjną gier hazardowych, ale jej poziom istotnie różni się pomiędzy grami, a podane wartości zależą przy

tym od wariantu zasad gry. Chciałbym zaznaczyć, że celem niniejszej pracy nie jest wykazanie, że agent może trwale uzyskać dodatnią wartość oczekiwaną w grze fundamentalnie zaprojektowanej na korzyść kasyna, lecz sprawdzenie, które metody uczenia przez wzmacnianie są w stanie nauczyć się strategii minimalizującej tę przewagę.

2.6. Strategie bazowe i ocena jakości strategii

Zdecydowałem się wprowadzić dwie strategie bazowe: agenta losowego (ang. *random agent*) i agenta regułowego (ang. *rule-based agent*). Te dwa agenty są dla mnie punktami odniesienia, pozwalającymi ocenić jakość strategii wyuczonej przez agenta uczącego się przez wzmacnianie. Ważne dla mnie jest porównanie agentów uczących się pomiędzy sobą, ale również względem strategii bazowych. Agent losowy wybiera jedną z legalnych akcji bez uwzględniania siły kart, stanowi prosty punkt odniesienia i umożliwia wstępną kontrolę działania infrastruktury eksperymentalnej. Agent regułowy wykorzystuje z góry określone heurystyki i reprezentuje bardziej wymagający punkt odniesienia.

Samo przewyższenie agenta losowego nie stanowi wystarczającego dowodu uzyskania jakościowej strategii, natomiast porównanie z agentem regułowym pozwoli mi ocenić, czy uczenie prowadzi do decyzji konkurencyjnych wobec jawnie zaprojektowanych zasad.

2.7. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) jest paradygmatem uczenia maszynowego, w którym agent uczy się podejmowania decyzji poprzez interakcję ze środowiskiem. W każdym kroku agent obserwuje aktualny stan (ang. *state*) środowiska, wybiera akcję (ang. *action*), a następnie otrzymuje nagrodę (ang. *reward*) oraz przechodzi do kolejnego stanu. Celem agenta jest wyuczenie strategii, określanej również jako polityka decyzyjna (ang. *policy*), która maksymalizuje sumę nagród w długim horyzoncie czasowym [5].

Przed flopem gracz zna jedynie swoje dwie karty, ale może wykonać najwyższy zakład Play. Po flopie posiada więcej informacji, lecz maksymalny zakład jest mniejszy. Po odkryciu wszystkich kart wspólnych zna już pełny board, ale może wykonać wyłącznie zakład 1x Ante albo spasować. Agent musi więc nauczyć się kompromisu pomiędzy ryzykiem, ilością informacji oraz potencjalną wypłatą.

2.7.1. Podstawowe pojęcia

Zachowanie agenta opisuje polityka, która może być deterministyczna, przypisując stanowi jedną akcję, albo stochastyczna, określając rozkład prawdopodobieństwa wyboru akcji. Jakość decyzji ocenia się na podstawie zwrotu epizodycznego, czyli sumy nagród uzyskanych do końca epizodu:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \quad (2.1)$$

gdzie $\gamma \in [0, 1]$ jest współczynnikiem dyskontowania, a T oznacza koniec epizodu. Współczynnik γ reguluje wagę nagród odległych w czasie względem nagród natychmiastowych [5].

Oczekiwany zwrot z danego stanu opisuje funkcja wartości stanu $V^\pi(s)$, natomiast oczekiwany zwrot po wykonaniu akcji w danym stanie opisuje funkcja wartości akcji $Q^\pi(s, a)$. Te funkcje pozwalają porównywać decyzje i stanowią podstawę w wielu metodach uczenia [5].

W trakcie uczenia agent musi równoważyć eksplorację (ang. *exploration*), czyli wypróbowywanie nowych akcji w celu zdobycia informacji, oraz eksploatację (ang. *exploitation*), czyli wybór akcji uznanych dotąd za najlepsze. Niewłaściwy balans między nimi prowadzi do niestabilnego przebiegu uczenia.

Funkcje wartości i polityki można reprezentować w postaci tablicowej, gdy przestrzeń stanów jest niewielka, albo za pomocą aproksymacji funkcji, na przykład sieci neuronowej, gdy przestrzeń stanów jest duża. Przestrzeń stanów Ultimate Texas Hold'em jest zbyt duża na podejście tablicowe, dlatego funkcję wartości akcji aproksymuję siecią neuronową, co opisuję szczegółowo w rozdziale dotyczącym zastosowanej metody uczenia [5].

Dwa elementy konfiguracji procesu uczenia mają dla mnie niebagatelne znaczenie są to: sposób reprezentacji obserwacji, decydujący o tym, jakie informacje trafiają na wejście agenta, oraz konstrukcja funkcji nagrody, przekształcająca finansowy wynik rozdania w sygnał uczący. To na analizie tych dwóch elementów skupiam się w niniejszej pracy.

2.8. Modelowanie UTH jako procesu decyzyjnego

Kończąc niniejszy rozdział, chciałbym podkreślić, że w Ultimate Texas Hold'em agent nie zna pełnego stanu środowiska, a jedynie jego obserwację. Agent nie wie, jakie karty ma krupier, ani jakie karty są następne w talii. Problem niniejszej pracy można więc traktować jako częściowo obserwowalny proces decyzyjny. To rozróżnienie obserwacji od pełnego stanu jest istotne dla zachowania niepełnej informacji, która jest charakterystyczna dla pokera. Odwzorowałem to w projekcie środowiska opisanym w kolejnym rozdziale [5].

3. Projekt i implementacja środowiska Ultimate Texas Hold'em

W niniejszym rozdziale przedstawiam projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Wcześniej opisane zasady gry odwzorowałem w postaci silnika, który jest odpowiedzialny za zarządzanie całym przebiegiem rozdania, udostępnia on agentowi legalne akcje do wykonania oraz rozlicza wynik finansowy rozdania.

Implementację podzieliłem na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Kodowanie obserwacji, odwzorowanie akcji i konstrukcja nagrody są poza warstwą realizującą zasady gry, co umożliwi mi porównywanie różnych reprezentacji stanu i funkcji nagrody.

3.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie (epizod). Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów.

Agent może podjąć decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play przez gracza środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera, i przechodzi do stanu końcowego.

Wartości zakładów wyraziłem w jednostkach względnych, tzn. jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Zdecydowałem się podejść do tego w taki sposób, aby uniezależnić środowisko od konkretnej waluty i wysokości stawki.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania, będący podstawą nagrody terminalnej. Rozdzieliłem mechanizm rozliczania gry od funkcji nagrody. Uzasadnienie tej decyzji przedstawiam w podrozdziale 3.8.

Zadbałem również o powtarzalność eksperymentów, przez co talia kart może być tasowana przy użyciu ziarna generatora liczb pseudolosowych, dzięki temu mogłem odtworzyć całą sekwencję rozdań. W testach jednostkowych zastosowałem dodatkowo talię deterministyczną, która pozwala określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

3.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowałem jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która odpowiada za koordynowanie działania pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjąłem taki podział, aby ograniczyć zależności pomiędzy poszczególnymi częściami systemu. Model kart i ewaluacja układów są dzięki temu testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych.

Moduł `uth` zawiera modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę menedżera tej warstwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje ona odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat danego kroku.

Agent nie komunikuje się bezpośrednio z talią, ewaluatorem ani pełnym stanem wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Wynikiem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie podejście pozwala mi rozgraniczyć logikę środowiska od implementacji agenta.

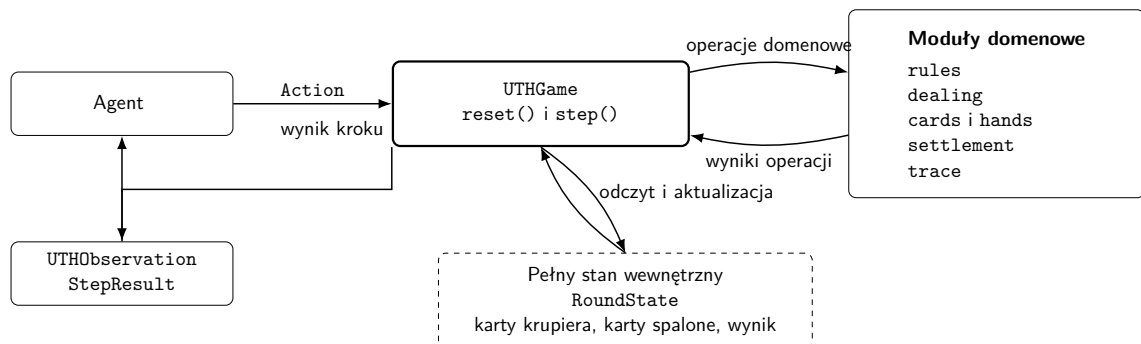
W tabeli 3.1 przedstawiłem najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Co istotne, silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmocnianie. Integrację z takimi bibliotekami wydzieliłem do osobnej warstwy korzystającej z publicznego interfejsu `UTHGame` i przekształcającej obiekty domenowe na reprezentacje numeryczne. Pozwala mi to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiłem na rysunku 3.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.

Tabela 3.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

Moduł	Odpowiedzialność
cards	Reprezentacja pojedynczej karty, jej rangi, koloru oraz talii składającej się z 52 unikalnych kart.
hands	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
uth.models	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
uth.rules	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
uth.dealing	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
uth.card_source	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
uth.settlement	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
uth.game	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
uth.trace	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.



Rys. 3.1. Architektura i przepływ informacji w środowisku UTH

3.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zaprojektowałem niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera, o czym wspominałem w rozdziale poświęconym możliwym drogom rozwoju.

3.3.1. Reprezentacja karty

Pojedynczą kartę zaimplementowałem jako niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zakodowałem za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Przez zastosowanie typów wyliczeniowych ograniczyłem możliwość utworzenia karty zawierającej niepoprawną wartość, a przez podejście do karty jako niemutowalnego obiektu zapewniłem bezpieczne umieszczanie jej w krotkach i zbiorach, a także możliwość wykorzystania porównania liczebności zbioru do wykrywania zduplikowanych kart.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Zdecydowałem się na taką konwencję ponieważ umożliwia mi to bezpośrednie porównywanie rang podczas oceny układów.

Warto dodać, że as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez ewaluator układów. Zbiór kolorów obejmuje trefle, karo, kiery i piki.

3.3.2. Standardowa talia

Klasa `Deck` reprezentuje standardową talię. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (3.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu talii i następnie ją usuwa z kolekcji. Jest również operacja służąca do pobrania wielu kart `draw_many()`. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba dostępnych kart pozostałych w talii powoduje zgłoszenie wyjątku. Chciałem, aby takie zachowanie umożliwiała szybkie wykrycie niepoprawnego przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Zadbalem o możliwość utworzenia talii w konkretnej kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart.

Aby zapewnić powtarzalność eksperymentów, w klasie `Deck` zastosowałem możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart. Na poziomie klasy `UTHGame` zastosowałem dodatkowo generator nadrzędny, który przy każdym rozpoczęciu nowego rozdania generuje osobne ziarno przekazywane do nowej talii. Dzięki temu kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne talie, co ogranicza wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

3.3.3. Abstrakcja źródła kart i talia testowa

Silnik rozdania nie zależy bezpośrednio od klasy `Deck`. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart, dzięki czemu moduł odpowiedzialny za rozdawanie nie musi rozpoznawać, czy korzysta z przetasowanej talii, czy ze źródła deterministycznego.

Dzięki deterministycznemu źródłu byłem w stanie przygotować powtarzalne testy obejmujące określone scenariusze rozgrywki, kwalifikacji krupiera i rozliczenia zakładów. Do testowania konkretnych scenariuszy wprowadziłem klasę `FixedDeck`, która jest jawnie przekazaną sekwencją kart. Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy każda karta jest unikalna, dzięki czemu błąd w danych testowych nie może doprowadzić do rozdania zawierającego dwie identyczne karty.

3.4. Ocena układów pokerowych

Zadaniem modułu oceny układów pokerowych jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W finalnej fazie podczas showdownu ewaluator jest wywoływany dla gracza i krupiera zawsze na siedmiu kartach. Pozostałe dwa warianty wykorzystuje natomiast agent regułowy opisany w rozdziale poświęconym agentom bazowym.

3.4.1. Reprezentacja wartości ręki

Do reprezentacji kategorii układów pokerowych podszedłem implementując typ wyczeniowy `HandRank`. Kolejnym kategoriom przypisuje rosnące wartości liczbowe od 0 do 8, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii odbywa się wprost przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków

(np. gracz i krupier mogą mieć układy tej samej kategorii, lecz różniące się rangą kart tworzących dany układ albo kartami dodatkowymi). Z tego względu pełną wartość ręki zaimplementowałem jako obiekt `HandValue`, zawierający kategorię układu `rank` oraz uporządkowaną krotkę wartości rozstrzygających remis `tiebreak`.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (3.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości kolejne elementy krotki `tiebreak`.

Dla lepszego zobrazowania, wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (3.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Zadbałem, aby obiekt `HandValue` sprawdzał zgodność długości krotki `tiebreak` z kategorią układu, walidował również, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. W ten sposób ograniczyłem możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

3.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje pięć unikalnych kart. W pierwszym kroku funkcja oblicza liczbę wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki: czy wszystkie karty mają ten sam kolor oraz czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najsłabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie funkcja sprawdza kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość `tiebreak` jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest jedynie nazwa układu, lecz kompletna wartość umożliwiająca porównanie z dowolną inną poprawną ręką. Szczególnym przypadkiem jest strit `A-2-3-4-5`, w którym as pełni rolę najniższej karty, ewaluator zwraca wtedy strita o najwyższej karcie równej 5, bez zmiany wartości asa w pozostałych kategoriach.

3.4.3. Wybór najlepszej ręki

Podczas showdownu gracz i krupier dysponują łącznie siedmioma kartami. Rękę pokerową tworzy dokładnie pięć kart, dlatego funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`. Generowanie każdego możliwego podzbioru nie jest optymalnym rozwiązaniem, jednak liczba kombinacji jest niewielka i stała, co potraktowałem jako kompromis pomiędzy prostotą implementacji a wydajnością.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (3.4)$$

Dla liczby kart obsługiwanych przez ewaluator jest tyle kombinacji:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (3.5)$$

W przypadku standardowego showdownu UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ. Uznałem to za wygodny sposób prezentacji wyniku showdownu oraz diagnostyki ewaluatora.

Dla najwyższej kategorii, jaką jest poker królewski, nie tworzyłem osobnej kategorii `HandRank`. Rozwinąłem go jako poker z asem jako najwyższą kartą.

3.5. Stan wewnętrzny i obserwacja agenta

Ponieważ Ultimate Texas Hold'em jest grą z niepełną informacją, odwzorowanie tej właściwości wymagało od mnie rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowałem dwa odrębne modele danych: `RoundState` przedstawiający pełny stan wewnętrzny rozdania oraz `UTHObservation` reprezentujący obserwację dostępną agentowi.

Agent nie powinien i nie otrzymuje odwołania do obiektu `RoundState`. Każda obserwacja jest tworzona na kanwie pełnego stanu przez osobną funkcję projekcji, która wyciąga ze stanu wyłącznie informacje dozwolone z perspektywy gracza.

3.5.1. Pełny stan rozdania

Obiekt `RoundState` zawiera wszystkie dane na temat rozdania. Są to:

- aktualną fazę gry,
- dwie karty własne gracza,
- dwie ukryte karty krupiera,
- odkryte karty wspólne,
- karty spalone,
- mnożnik wykonanego zakładu Play,
- końcowy wynik rozdania,
- szczegółowe rozliczenie zakładów,
- rezultat showdownu.

Naturalnie nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat showdownu oraz mnożnik zakładu Play pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Dla przykładu liczba kart wspólnych i spalonych jest zależna od aktualnej fazy. Tę zależność przedstawiam w tabeli 3.2.

Tabela 3.2. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone fol-dem albo showdownem.

Obiekt stanu zaimplementowałem jako niemodyfikowalny, co wymusza utworzenie nowej instancji RoundState przy każdym przejściu środowiska (zamiast modyfikowania istniejącego obiektu). Zdecydowałem się na takie rozwiązanie, żeby ograniczyć ryzyko, że obiekt stanu może potencjalnie zostać niewłaściwie zmodyfikowany, ale też takie rozwiązanie ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

- poprawny typ fazy,
- obecność dokładnie dwóch kart gracza i dwóch kart krupiera,
- liczba kart wspólnych właściwa dla danej fazy,

- liczba kart spalonych właściwa dla danej fazy,
- brak zduplikowanych kart w całym stanie,
- zgodność pól końcowych z fazą rozdania.

Sprawdzone jest również, aby stan niekońcowy nie zawierał wyniku ani rozliczenia, stan zakończony foldem nie zawierał mnożnika Play ani rezultatu showdownu, natomiast stan zakończony showdownem powinien zawierać zarówno mnożnik wykonanego zakładu, jak i szczegóły ocenionych rąk.

3.5.2. Obserwacja agenta

Obiekt `UTHObservation` zawiera wyłącznie te dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- aktualna faza gry,
- dwie karty własne gracza,
- dotychczas odkryte karty wspólne,
- zbiór legalnych akcji,
- mnożnik zakładu Play w obserwacji terminalnej.

W ramach tego obiektu umieściłem zbiór legalnych akcji, które agent może podjąć w danej fazie, dzięki temu zdejmuję z agenta konieczność odtwarzania zasad gry na podstawie fazy, co ułatwia implementację agentów bazowych.

Obserwacja `UTHObservation`, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry.

3.5.3. Projekcja stanu na obserwację

Obiekt obserwacji dla agenta jest tworzony przez funkcję projekcji.

$$O_t = \phi(S_t), \quad (3.6)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla warunków mojego środowiska funkcja projekcji ma postać

$$\phi(S_t) = (p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t), \quad (3.7)$$

gdzie:

- p_t oznacza aktualną fazę gry,
- C_t^{player} oznacza karty własne gracza,
- $C_t^{community}$ oznacza odkryte karty wspólne,
- A_t^{legal} oznacza zbiór legalnych akcji,
- m_t oznacza mnożnik zakładu Play, jeżeli został już wykonany.

Karty krupiera i karty spalone należą do S_t , natomiast kolejność kart w talii jest przechowywana przez wewnętrzne źródło kart. Żadna z tych informacji nie jest elementem O_t , dokładny zakres pól obu obiektów przedstawiłem już w podrozdziale 3.5.1 i 3.5.2.

Ważne jest, że wynik rozdania, rozliczenie i szczegóły showdownu są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Z racji, że te dane pojawiają się dopiero po wykonaniu ostatniej decyzji, nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

3.6. Maszyna stanów i przestrzeń akcji

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowałem jako maszynę stanów. Przejście do następnej fazy jest wyznaczone przez bieżącą fazę i akcję, z kolei zawartość stanu, tym samym obserwacji kolejnego stanu zależy od kart zwróconych przez źródło. Bieżąca faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji.

Przebieg rozdania można przedstawić jako sekwencję faz:

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (3.8)$$

Faza TERMINAL jest stanem absorbującym, tzn. nie ma dla niej żadnych legalnych akcji, a rozdanie wchodzi w nią wyłącznie w wyniku spasowania albo wykonania zakładu Play zakończonego showdownem.

Metoda `reset()`, wywoływana na początku każdego rozdania, rozdaje po dwie karty graczowi i krupierowi w kolejności P_1, D_1, P_2, D_2 , gdzie P to gracz a D to krupier, tworzy stan PREFLOP i zwraca pierwszą obserwację. Karty wspólne i spalone nie są jeszcze dobierane. Dzieje się to w odpowiedniej fazie. Każde poprawne wykonanie metody `step()` przesuwa rozdanie do kolejnej fazy albo do stanu terminalnego. Agent nie może pozostać w tej samej fazie po wykonaniu akcji.

Pełna przestrzeń akcji składa się z sześciu decyzji domenowych:

$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}, \quad (3.9)$$

Maszyna stanów z podziałem na fazy i zbiory legalnych akcji wygląda następująco:

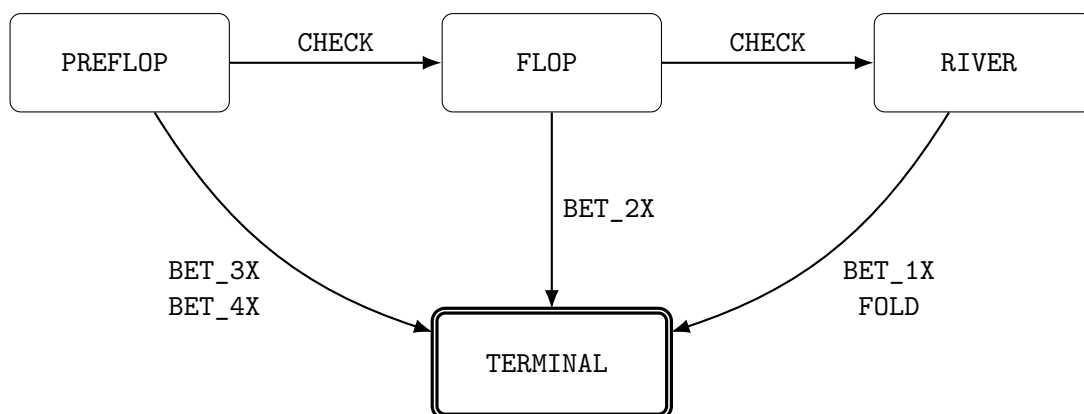
$$\mathcal{A}_{legal}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (3.10)$$

Akcja CHECK odkłada decyzję o zakładzie Play i przesuwa rozdanie do następnej fazy, spalając kartę oraz odkrywając flop albo turn i river. Każda akcja typu BET ustala mnożnik Play wynikający z fazy: $4\times$ lub $3\times$ przed flopem, $2\times$ po flopie i $1\times$ na riverze.

Po zakładzie Play środowisko odkrywa pozostałe karty wspólne, przeprowadza showdown i przechodzi do stanu terminalnego. Akcja FOLD kończy rozdanie na riverze bez showdownu.

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz w trakcie rozdania: każda akcja typu BET prowadzi bezpośrednio do stanu terminalnego, a akcja CHECK jedynie przesuwa decyzję do późniejszej fazy.

Dla zilustrowania przebiegu rozdania posłużę się schematem przedstawionym na rysunku 3.2.



Rys. 3.2. Maszyna stanów pojedynczego rozdania UTH

3.7. Showdown i rozliczanie zakładów

Showdown jest fazą końcową każdego rozdania, w którym gracz wykonał zakład Play. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. W przypadku kiedy gracz spsuje, nie dochodzi do showdownu.

3.7.1. Przebieg showdownu

Przypomnę, że podczas showdownu gracz i krupier dysponują siedmioma kartami. Ewaluator jest odpowiedzialny za wybór najlepszego pięciokartowego układu osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie ShowdownResult,

który zawiera:

- najlepszą rękę gracza,
- najlepszą rękę krupiera,
- wynik porównania z perspektywy gracza,
- informację o kwalifikacji krupiera.

Zbiór wszystkich możliwych wyników showdownu przedstawiam następująco:

$$\mathcal{O}_{showdown} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (3.11)$$

Konkretny rezultat porównania rąk gracza i krupiera jest wyznaczany przez funkcję O : dla wartości rąk H_p oznaczającej rękę gracza oraz H_d oznaczającej rękę krupiera, wynik showdownu można zdefiniować jako:

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (3.12)$$

Wydawać by się mogło, że w momencie kiedy krupier się nie kwalifikuje, nie ma sensu porównywać rąk obu stron. Rozważę jednak przykład, gdzie każda ze stron ma jedynie wysokie karty (najniższy możliwy układ), ale najwyższa karta krupiera jest wyższa od najwyższej karty gracza. W takim przypadku krupier wygrywa rozdanie, mimo że się nie kwalifikuje. Dlatego porównuję rękę gracza i krupiera nawet w sytuacji, kiedy krupier się nie kwalifikuje.

3.7.2. Kwalifikacja krupiera

W Ultimate Texas Hold'em krupier kwalifikuje się, jeżeli posiada co najmniej jedną parę, taki warunek można zapisać w formie równania:

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (3.13)$$

gdzie $\text{rank}(H_d)$ oznacza rangę układu krupiera.

Samo zjawisko kwalifikacji krupiera nie zmienia wyniku porównania rąk, jednakże wpływa na sposób rozliczenia zakładu Ante, ponieważ:

- jeżeli krupier kwalifikuje się, zakład Ante wygrywa lub przegrywa zgodnie z wynikiem showdownu,

- jeżeli krupier nie kwalifikuje się, zakład Ante jest zwracany niezależnie od tego, która strona ma silniejszą rękę.

Odnosnie zakładu Play jest on rozliczany niezależnie od kwalifikacji krupiera w wyniku bezpośredniego porównania rąk. Z kolei zakład Blind jest rozliczany według układu gracza (w przypadku jego zwycięstwa), przy remisie zwracany, a przy przegranej gracza jest przegrywany.

Wszystkie wyżej opisane zależności w celu ułatwienia przedstawiam w tabeli 3.3.

Tabela 3.3. Rozliczenie zakładów po showdownie

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

3.7.3. Model rozliczenia zakładów

Zaprojektowałem obiekt `WagerSettlement`, który reprezentuje rozliczenie pojedynczego zakładu. Składa się on z `stake`, czyli początkowej wartości postawionego zakładu, oraz `net_profit`, czyli zysku albo straty netto.

Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi $g = s + n$, jeśli $g = s$ oznacza to zwrot stawki, w przypadku $g = 0$ jej pełną utratę, a stawka $s = 0$ zakład niepostawiony.

Pełne rozliczenie rozdania reprezentuje obiekt `Settlement`, zawierający osobne wyniki dla Ante, Blind oraz Play. Łączny wynik netto S_{net} jest sumą zysków netto wszystkich trzech zakładów, można go zapisać w postaci równania:

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}. \quad (3.14)$$

Do reprezentacji wartości finansowych używam typu `Fraction`, ponieważ ten typ pozwala na przechowywanie wypłaty takiej jak $3/2$ (bez stosowania przybliżeń zmiennoprzecinkowych), w taki sposób unikam błędów zaokrągleń.

3.7.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W sytuacji kiedy wygrywa gracz, zysk z racji tego zakładu jest uzależniony od tego, jak wysoki układ uda się graczowi osiągnąć. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 3.4.

Zakład Blind jest stosowany wyłącznie wtedy, gdy gracz wygrał showdown. Przy

Tabela 3.4. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

remisie zakład jest zwracany, natomiast przy wygranej krupiera pełna stawka Blind zostaje utracona.

3.7.5. Rozliczenie Ante, Play i spasowania

Zakład Ante przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem showdownu. W przypadku braku kwalifikacji zakład jest zwracany.

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Żeby lepiej to zobrazować, posłużę się przykładem, powiedzmy, że gracz postawił wygrany zakład Play $4\times$, co daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

Jeżeli gracz spasuje na riverze, nie dochodzi do showdownu. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie folda ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0). \quad (3.15)$$

Wtedy łączna stawka wynosi dwie jednostki, a wynik netto jest równy -2 .

W celu rozróżnienia zakładu niepostawionego od zakładu postawionego, ale zwróconego, w rozliczeniu Play zapisywana jest stawka równa zero.

3.8. Funkcja nagrody i wynik epizodu

Aby móc sprawnie zmieniać warianty funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmocnianie, oddzieliłem mechanizm rozliczania zakładów od konstrukcji sygnału nagrody. Silnik domenowy wyznacza dokładny wynik finansowy rozdania zgodny z zasadami Ultimate Texas Hold'em. Natomiast konstruowanie sygnału nagrody na jego podstawie należy do warstwy integrującej silnik z agentem. Dzięki temu mogę w prosty sposób porównywać różne funkcje nagrody, bez ingerencji w zasady gry.

3.9. Interfejs silnika

Silnik domenowy zaimplementowałem w taki sposób, aby udostępniał niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia, co czyni go dość uniwersalnym rozwiązaniem do wykorzystania przy różnych eksperymentach.

Fundamentalnym elementem jest klasa `UTHGame`. Pojedyncza instancja tej klasy przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

3.9.1. Tworzenie silnika i rozpoczęcie epizodu

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry: `seed` oraz `record_history`. Ziarno określa powtarzalny strumień kart w talii generowanych dla kolejnych rozdawań, nie ustala jednak pojedynczej, niezmienniej talii dla całej instancji. Rejestrowanie historii jest domyślnie wyłączone, służy ono do zbierania dodatkowych obiektów diagnostycznych.

Każde nowe rozdanie rozpoczyna się właśnie wywołaniem metody `reset()`, które zwraca pierwszą obserwację w fazie `PREFLOP`. W standardowym użyciu środowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Rozpoczęcie rozdania rozumiem jako wybór lub utworzenie źródła kart, rozdanie po dwie karty graczowi i krupierowi, utworzenie stanu `PREFLOP`, wyzerowanie historii bieżącego rozdania oraz utworzenie pierwszej obserwacji. Reset jest wykonywany transakcyjnie, silnik najpierw spróbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po powodzeniu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

3.9.2. Wykonanie decyzji

Zaprojektowałem metodę `step()` tak, aby przyjmowała jedną wartość typu `Action`, wykonała jej odpowiadające przejście i zwracała obiekt `StepResult`, zawierający:

- obserwację po wykonaniu akcji,
- wynik rozdania (jeśli osiągnięto stan terminalny),
- rozliczenie zakładów w stanie terminalnym,
- szczegóły showdownu (jeśli gracz nie spasował).

Wynik metody `step()` kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera informacje niezbędne do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy, która wchodzi w skład obserwacji.

Choć zbiór legalnych akcji jest wyznaczany przez środowisko i umieszczany w obserwacji, metoda `step()` nie ufa bezkrytycznie przekazanej decyzji. Każda akcja jest walidowana ponownie przed jakąkolwiek zmianą stanu. Walidacja następuje przed dobieraniem kart, więc odrzucona decyzja nie zużywa kart z talii, nie zmienia fazy i nie jest dodawana do historii.

3.9.3. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 3.5.

Tabela 3.5. Publiczny interfejs klasy `UTHGame`

Element	Typ wyniku	Odpowiedzialność
<code>reset()</code>	<code>UTHObservation</code>	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
<code>step(action)</code>	<code>StepResult</code>	Wykonuje legalną akcję i zwraca wynik przejścia.
<code>observation</code>	<code>UTHObservation</code>	Zwraca aktualne informacje widoczne dla agenta.
<code>state</code>	<code>RoundState</code>	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
<code>is_started</code>	<code>bool</code>	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
<code>history_enabled</code>	<code>bool</code>	Informuje, czy rejestrowanie historii jest aktywne.
<code>trace</code>	<code>RoundTrace</code> lub <code>None</code>	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta. Udostępniłem ją jedynie do celów testowych i diagnostycznych. Agent otrzymuje wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

3.10. Rejestrowanie przebiegu i walidacja środowiska

Bardzo ważnym elementem projektu środowiska jest pokrycie jego implementacji testami, ponieważ potencjalne niedopatrzenia na etapie implementacji, skrajne przypadki, błędy w logice bezpośrednio rzutują na wyniki eksperymentów, co może doprowadzić do nieefektywnego uczenia się agentów. Aby zapobiec takiemu scenariuszowi,

dużą uwagę poświęciłem testom jednostkowym i integracyjnym, wprowadziłem opcjonalny mechanizm rejestrowania przebiegu rozdania oraz wzbogaciłem testy o weryfikację całych komponentów jak i ścieżek rozgrywki.

3.10.1. Rejestrowanie historii i jej rekordy

Rejestrowanie historii jest aktywowane parametrem `record_history` przekazywanym podczas tworzenia obiektu `UTHGame`. Domyślnie mechanizm ten pozostaje wyłączony, aby podczas uczenia nie generować dodatkowych obiektów diagnostycznych, które nie są potrzebne agentowi i zajmują pamięć oraz czas. Po jego włączeniu każde poprawnie rozpoczęte rozdanie otrzymuje dodatni, rosnący identyfikator `round_id`, a lista decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`. Właściwość `trace` zwraca obiekt `RoundTrace`. Jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`. Mechanizm ten nie stanowi pamięci agenta i nie jest częścią obserwacji. Wykorzystuję go do odtwarzania przebiegu wybranych rozdań w celu weryfikacji zgodności decyzji agenta z legalną przestrzenią akcji.

Niemutowalny obiekt `DecisionRecord` przechowuje pojedynczą decyzję agenta: obserwację dostępną agentowi przed wykonaniem decyzji oraz wybraną akcję. Argumentem za zapisywaniem obserwacji sprzed przejścia jest to, że chciałem zachować informację, dla jakiej obserwacji agent podjął konkretną decyzję, zapisywanie obserwacji po wykonaniu akcji byłoby w mojej ocenie niejednoznaczne, ponieważ taki stan zawierałby informacje już odkrytych kart, które mogłyby być skutkiem wykonanej akcji. Podczas tworzenia rekordu sprawdzane jest, czy akcja należy do zbioru legalnego dla tej obserwacji.

Cały przebieg rozdania opisuje niemodyfikowalny obiekt `RoundTrace`, który zawiera identyfikator rozdania, uporządkowaną krotkę rekordów decyzji oraz aktualny pełny stan `RoundState`. Obserwacje w rekordach zawierają wyłącznie informacje dostępne agentowi, natomiast końcowy stan jest pełnym obiektem diagnostycznym, z tego powodu `RoundTrace` nie jest przekazywany agentowi, lecz służy testom.

Zapis historii jest transakcyjny. Metoda `step()` zapamiętuje bieżącą obserwację, wykonuje walidację i przejście, a rekord dodaje dopiero po utworzeniu poprawnego kolejnego stanu. Jeżeli walidacja lub odkrywanie kart zakończy się wyjątkiem, rekord nie powstaje, więc historia nie zawiera akcji nielegalnych, decyzji po zakończeniu rozdania ani niedokończonych przejść. Podobny cel mi przyswiecał przy metodzie `reset()`, która zapisuje nowy stan dopiero po rozdaniu wszystkich czterech kart początkowych.

3.10.2. Poziomy testowania

Wspominałem, że testy są kluczowe, dlatego uporządkowałem je według odpowiedzialności komponentów. Testy jednostkowe sprawdzają w izolacji model kart i talię, ewaluator, modele stanu, rozdawanie, legalność akcji, showdown oraz rozliczenia. Testy

integracyjne z kolei wykonują kompletne rozdania obejmujące wszystkie ścieżki maszyny stanów, od `reset()` do stanu terminalnego, wraz z rejestrowaniem historii.

Testy weryfikują również między innymi brak powtarzających się kart, zgodność liczby kart wspólnych i spalonych z fazą, brak danych terminalnych w stanie pośrednim, brak showdownu po spasowaniu, wykonanie co najwyżej jednego zakładu Play, brak kart krupiera i kart spalonych w obserwacji oraz brak zmiany stanu po odrzuconej akcji. Walidacja obejmuje także przypadki brzegowe i próby utworzenia stanów naruszających reguły domenowe.

W przypadku realizacji testów postawiłem na bibliotekę `pytest`. Podczas prac nad silnikiem analizowałem na bieżąco zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło mi to wykryć nieprzetestowane ścieżki walidacji oraz przypadki brzegowe ewaluatora.

3.11. Ograniczenia środowiska

Choć uważam, że zaprojektowany i zaimplementowany przeze mnie model środowiska Ultimate Texas Hold'em jest wystarczający do przeprowadzenia badań nad strategiami podejmowania decyzji, to nie odwzorowuje on wszystkich aspektów gry w kasynie.

Prawdą jest, że zakłady Trips i Progressive nie wprowadzają dodatkowego momentu decyzyjnego, ponieważ ich wynik nie zależy od decyzji związanej z akcją Play. Niemniej zakłady poboczne wpływają na rozkład końcowych wypłat i utrudniałyby interpretację jakości strategii, a Progressive wymagałby dodatkowo modelu puli utrzymywanej pomiędzy rozdaniem, co naruszałoby niezależność epizodów. Oba zakłady poboczne można zaimplementować jako rozszerzenie warstwy rozliczeń, co może stanowić przedmiot dalszych badań.

Silnik nie modeluje również salda gracza, sesji, ryzyka bankructwa ani zarządzania kapitałem, a wartości zakładów wyraża w jednostkach Ante, dzięki czemu strategię można porównywać niezależnie od nominału.

Zaprojektowałem środowisko, które reprezentuje pojedynczego gracza wobec krupiera i nie symuluje pozostałych miejsc przy stole, a zakryte karty innych graczy i tak byłyby nieznane, więc rozkład kart widocznych dla gracza odpowiadałby losowaniu bez zwracania ze standardowej talii.

Obserwacja jest obiektem domenowym zawierającym karty, fazę i zbiór legalnych akcji. W tej postaci jest czytelna i wygodna w testach, lecz nie może być bezpośrednio podana modelom uczenia maszynowego. Albowiem kodowanie `UTHObservation` do reprezentacji numerycznej pozostaje poza zakresem klasy `UTHGame`, ponieważ porównanie reprezentacji stanu jest jednym z elementów badawczych pracy. Wspólny silnik jest wygodny, bo pozwala zestawiać różne reprezentacje na identycznych rozdaniach i regułach.

3.12. Podsumowanie

W niniejszym rozdziale przedstawiłem projekt oraz implementację pojedynczego rozdania Ultimate Texas Hold'em. Silnik podzieliłem na niezależne moduły odpowiedzialne za reprezentację kart, ocenę układów, rozdawanie, przechowywanie stanu, walidację akcji, showdown oraz rozliczanie zakładów, a przebieg rozdania odwzorowałem jako maszynę stanów obejmującą fazy PREFLOP, FLOP, RIVER i TERMINAL, w której każda akcja BET kończy część decyzyjną i wymusza co najwyżej jeden zakład Play.

Ważnym elementem projektu jest rozdzielenie pełnego stanu RoundState od obserwacji UTHObservation, mianowicie agent otrzymuje wyłącznie własne karty, odkryte karty wspólne, fazę i legalne akcje, podczas gdy karty krupiera, karty spalone i kolejność talii pozostają wewnętrzne. Showdown korzysta z ogólnego ewaluatora wybierającego najlepsze pięć kart z siedmiu, a rozliczenia Ante, Blind i Play są przechowywane dokładnie za pomocą typu Fraction. Deterministyczne źródło FixedDeck oraz opcjonalny mechanizm historii zapewniają powtarzalność i możliwość analizy przebiegu rozdań.

Na tym etapie dostarczyłem silnik gry, który zapewnia podstawę do implementacji agentów bazowych, infrastruktury eksperymentalnej oraz metod uczenia przez wzmocnienie analizowanych w dalszej części pracy.

4. Agenci bazowi i infrastruktura eksperymentalna

W tym rozdziale przedstawiam projekt agentów bazowych oraz infrastruktury służącej do uruchamiania i oceny strategii w środowisku Ultimate Texas Hold'em. Rolę agentów bazowych jako punktów odniesienia dla metod uczenia przez wzmacnianie opisałem w rozdziale 2.

Zaprojektowałem i zaimplementowałem wspólny kontrakt agenta, mechanizm pojedynczego epizodu i symulację wielu rozdań, agenta losowego oraz agenta regułowego. Infrastrukturę uzupełniłem o mechanizm zbierania metryk i zapis wyników eksperymentów. Wszystkie te elementy korzystają z publicznego interfejsu środowiska, o którym pisałem w rozdziale 3.

4.1. Wspólny interfejs agenta

Zdefiniowałem wspólny kontrakt `Agent`, który określa, w jaki sposób agent komunikuje się ze środowiskiem. Kontrakt ten udostępnia metodę `select_action()`, która przyjmuje obiekt `UTHObservation` i zwraca podjętą przez agenta decyzję typu `Action`.

Wykorzystanie protokołu do konstrukcji kontraktu pozwala mi zdefiniować agenta w sposób elastyczny, klasa agenta nie musi dziedziczyć po wspólnej klasie bazowej, lecz musi udostępniać metodę o wymaganej sygnaturze. Dzięki temu ten sam mechanizm prowadzenia rozgrywki mogą wykorzystać dla agenta losowego, regułowego i uczącego się przez wzmacnianie bez znajomości ich wewnętrznej implementacji.

4.2. Prowadzenie pojedynczego epizodu

W celu sprawnego przeprowadzenia eksperymentu zaimplementowałem osobny moduł, który nazwałem `runnerem`. Zdefiniowałem w nim funkcję `play_round()`, która łączy agenta z silnikiem `UTHGame`. Funkcja ta rozpoczyna rozdanie, przekazuje bieżącą obserwację agentowi, wykonuje wybraną akcję i powtarza ten proces do osiągnięcia stanu terminalnego.

Każda decyzja zaakceptowana przez środowisko zostaje zapisana w kolejności wykonania. Po zakończeniu rozdania funkcja zwraca obiekt `EpisodeResult`, który zawiera sekwencję akcji, wynik rozdania oraz pełne rozliczenie zakładów Ante, Blind i Play.

Model `EpisodeResult` udostępnia również wynik netto, łączną wartość postawionych zakładów, liczbę decyzji i mnożnik zakładu Play. Stanowi w ten sposób lekką reprezentację zakończonego epizodu.

4.3. Agent losowy

Najprostszą strategię bazową stanowi agent losowy. Nie analizuje on wartości kart ani prawdopodobieństwa wygranej, a dla każdej obserwacji wybiera jedną z legalnych ak-

cji z jednakowym prawdopodobieństwem. W obserwacji `UTHObservation` agent dostaje zbiór legalnych akcji do wykonania. Akcje te są porządkowane zgodnie z kolejnością wartości enuma `Action`, przez co wynik generatora pseudolosowego nie zależy od kolejności iterowania po zbiorze.

Środowisko i agent posiadają niezależne generatory pseudolosowe. Generator środowiska odpowiada za kolejność kart, natomiast generator agenta odpowiada wyłącznie za wybór akcji. Rozdzielając źródła losowości, uzyskałem możliwość zmiany strategii agenta, na przykład jego seedu albo całej implementacji, bez zmiany przebiegu rozdań. Dzięki temu, jak pokazuję w sekcji 4.7, mogę porównywać agenta losowego i agenta regułowego na dokładnie tym samym harmonogramie talii.

Napisałem testy obejmujące zgodność ze wspólnym protokołem, wybieranie wyłącznie legalnych akcji, powtarzalność decyzji dla tego samego seedu oraz obsługę niepoprawnych danych wejściowych. Jest również test potwierdzający, że agent nie podejmuje decyzji dla obserwacji terminalnej. Testy samego runnera opisuję w sekcji 4.9.

4.4. Agent regułowy

Agent losowy jest podstawowym punktem odniesienia, lecz nie korzysta w ogóle z wiedzy o grze. Dlatego w celu uzyskania lepszego punktu odniesienia zdefiniowałem agenta regułowego, który w przeciwieństwie do agenta losowego podejmuje decyzje na podstawie kart widocznych w obserwacji oraz aktualnej fazy rozdania. Przyjęte reguły oparłem na uproszczonej strategii Ultimate Texas Hold'em opublikowanej przez Wizard of Odds [2].

4.4.1. Reguły decyzyjne

Zanim zostaną odkryte karty wspólne, agent podejmuje decyzję o zakładzie Play w wysokości czterokrotności Ante dla układów początkowych przedstawionych w tabeli 4.1.

Tabela 4.1. Układy początkowe uprawniające do zakładu Play 4× przed flopem

Układ początkowy	Warunek
Para	od trójek wzwyż
As	z dowolną kartą towarzyszącą
Król	z dowolną kartą w tym samym kolorze
Król	z kartą pięć lub wyższą w różnych kolorach
Dama	z kartą sześć lub wyższą w tym samym kolorze
Dama	z kartą osiem lub wyższą w różnych kolorach
Walec	z kartą osiem lub wyższą w tym samym kolorze
Walec	z dziesiątką w różnych kolorach

Dla pary dwójek oraz wszystkich pozostałych układów agent wykonuje akcję CHECK. Ta strategia nie wykorzystuje zakładu `BET_3X`.

Jeżeli agent nie wykonał zakładu przed flopem, po odkryciu trzech kart wspólnych wybiera `BET_2X`, gdy posiada dwie pary lub silniejszy układ, ukrytą parę z wyjątkiem

pary dwójek albo cztery karty do koloru z własną kartą tego koloru o randze co najmniej dziesięć.

Przez ukrytą parę rozumiem parę wykorzystującą co najmniej jedną z dwóch prywatnych kart agenta. Jeżeli żaden z tych warunków nie jest spełniony, agent wykonuje akcję CHECK.

W ostatnim punkcie decyzyjnym agent wykonuje zakład BET_1X, jeżeli posiada ukrytą parę lub silniejszy układ. Dla słabszych układów wykorzystywana jest reguła oparta na liczbie kart mogących poprawić układ krupiera (ang. *outs*) do układu pokonującego gracza. Jeżeli liczba takich kart jest mniejsza niż 21, agent wykonuje BET_1X. W przeciwnym przypadku wybiera FOLD [2].

Jest to strategia deterministyczna, mówiąc prościej dla tej samej obserwacji agent zawsze zwraca tę samą akcję. Nie jest to jednak strategia optymalna, ponieważ opiera się na uproszczonych heurystykach szerzej opisanych przez Wizard of Odds, a nie na pełnym przeszukaniu przestrzeni decyzji.

4.4.2. Implementacja strategii regułowej

W celu implementacji agenta regułowego zdefiniowałem klasę RuleBasedAgent, która implementuje interfejs agenta pokerowego. Oddzieliłem reguły decyzyjne od klasy agenta, co pozwala mi na ich łatwe modyfikowanie w razie potrzeby.

Funkcje `should_raise_preflop()`, `should_raise_flop()` oraz `should_raise_river()` odpowiadają za ocenę obserwacji w poszczególnych fazach rozdania.

Do podjęcia decyzji na riverze zaimplementowałem funkcję `count_dealer_outs()`. Tworzy ona zbiór kart niewidocznych dla gracza, a następnie każdą z nich analizuje jako pojedynczą potencjalną kartę prywatną krupiera. Jeżeli dołączenie danej karty do kart wspólnych prowadzi do układu silniejszego od najlepszego układu agenta, karta zostaje zaklasyfikowana jako out.

4.5. Symulacja wielu epizodów i jej powtarzalność

Aby sprawnie przeprowadzać eksperymenty, składające się z wielu rozdań, zdefiniowałem obiekty `SimulationConfig` i `SimulationResult`.

Konfiguracja symulacji przechowuje uporządkowaną sekwencję seedów wykorzystywanych do tworzenia talii. Liczba seedów określa jednocześnie liczbę rozgrywanych epizodów.

Funkcja `run_simulation()` dla każdego seedu tworzy osobną talię oraz nową instancję środowiska `UTHGame`. Następnie wykorzystuje wspomnianą funkcję `play_round()` do rozegrania pełnego epizodu. Wyniki wszystkich rozdań są przechowywane w obiekcie `SimulationResult`.

W taki sposób oddzieliłem konfigurację eksperymentu od implementacji konkretnego agenta. Ten sam obiekt `SimulationConfig` może zostać wykorzystany do oceny

wielu różnych strategii.

Przy projektowaniu infrastruktury eksperymentalnej przyświecał mi cel łatwego odtwarzania przeprowadzonych symulacji. Dla każdego epizodu zapisuję seed talii, dzięki temu ponowne utworzenie talii z tym samym seedem daje tę samą kolejność kart. Było to dla mnie istotne, ponieważ kiedy porównuję agentów między sobą, mam pewność, że różnice w wynikach wynikają z ich strategii, a nie z losowego układu kart.

4.6. Metryki oceny

Aby móc porównywać skuteczność agentów między sobą, potrzebuję metryk umożliwiających ocenę jakości strategii. Miary, które zdefiniowałem poniżej nie są specyficzne dla agentów bazowych, ponieważ wykorzystuję je również do oceny agentów uczących się w kolejnych rozdziałach pracy. Za podstawową miarę jakości strategii uznałem wynik netto wyrażony w jednostkach zakładu Ante. I tak dla symulacji składającej się z N epizodów empiryczną wartość oczekiwaną oszacowałem jako średni wynik netto na jedno rozdanie:

$$\widehat{EV} = \frac{1}{N} \sum_{i=1}^N r_i, \quad (4.1)$$

gdzie r_i oznacza wynik netto i -tego epizodu wyrażony w jednostkach Ante.

Oprócz średniego wyniku obliczam również całkowity wynik netto oraz średnią i całkowitą wartość postawionych zakładów. Zmienność wyników oszacowałem za pomocą odchylenia standardowego próby:

$$s = \sqrt{\frac{\sum_{i=1}^N (r_i - \bar{r})^2}{N - 1}}. \quad (4.2)$$

Na podstawie odchylenia standardowego próby wyznaczyłem błąd standardowy średniej:

$$SE = \frac{s}{\sqrt{N}}. \quad (4.3)$$

Poza tymi miarami, do oceny jakości strategii dołączam również końcowe wyniki rozdań oraz częstotliwość wykonywania poszczególnych akcji.

4.7. Porównanie agentów bazowych

W celu zweryfikowania procesu eksperymentalnego, porównałem agenta losowego i agenta regułowego. Każdy z tych agentów rozegrał 10 000 rozdań. Seed generatora budującego harmonogram seedów talii wynosił 20260815, natomiast generator decyzji agenta losowego zainicjalizowano seedem 20260816. Sam obiekt UTHGame nie otrzymuje własnego seedu, powtarzalność zapewnia wyłącznie jawnie przekazana talia Deck(seed). Uzyskane podstawowe metryki przedstawiam w tabeli 4.2.

Tabela 4.2. Wyniki agentów bazowych dla 10 000 rozdań

Agent	$\widehat{EV}$	s	SE	Śr. stawka	Wynik netto
RandomAgent	-0,46435	4,46422	0,04464	4,7395	-4643,5
RuleBasedAgent	0,03865	4,08130	0,04081	4,1905	386,5

Wynika z tego, że agent losowy uzyskał średni wynik $-0,46435$ jednostki Ante na rozdanie, a agent regułowy osiągnął w tej samej próbie średni wynik $0,03865$, czyli wynik wyższy o około $0,503$ jednostki Ante na rozdanie. Jednocześnie średnia wartość środków stawianych przez agenta regułowego była niższa.

Pomimo tego, że agent regułowy uzyskał dodatni wynik w tej konkretnej próbie, nie interpretuję tego jako dowodu dodatniej rzeczywistej wartości oczekiwanej strategii. Błąd standardowy jego średniego wyniku wyniósł około $0,04081$, czyli wartość zbliżoną do samego oszacowania $\widehat{EV}$. Ten wynik eksperymentu służy jedynie jako punkt odniesienia dla późniejszej oceny agentów uczących się.

W celu zobrazowania wyników końcowych agentów bazowych przedstawiam w tabeli 4.3 rozkład wyników rozdań.

Tabela 4.3. Rozkład wyników agentów bazowych dla 10 000 rozdań

Agent	Wygrana	Wygrana krupiera	Fold	Remis
RandomAgent	45,01%	42,75%	8,49%	3,75%
RuleBasedAgent	47,92%	31,75%	16,79%	3,54%

Agent regułowy częściej kończył rozdania foldem, lecz znacznie rzadziej doprowadzał do rozstrzygnięcia zakończonego wygraną krupiera.

4.8. Zapis wyników eksperymentu

Zdecydowałem się na zapis wyników eksperymentu w dwóch formatach: JSON i CSV. Plik JSON zawiera konfigurację eksperymentu oraz zagregowane metryki obu agentów. Plik CSV przechowuje dane poszczególnych epizodów, w tym indeks rozdania, seed talii, nazwę agenta, wynik netto, całkowitą stawkę, wynik końcowy i sekwencję wykonanych akcji.

Taki podział na dwa osobne pliki pozwala mi zestawiać wyniki obu agentów dla tych samych seedów talii oraz dosyć łatwo tworzyć wizualizacje.

4.9. Walidacja infrastruktury eksperymentalnej

Pokryłem testami zarówno scenariusze pojedynczych epizodów, jak i symulacje wielu rozdań. Na poziomie pojedynczego epizodu testami runnera objąłem wszystkie legalne ścieżki zakończenia rozdania. Na poziomie symulacji sprawdziłem zgodność liczby epizodów z konfiguracją, powtarzalność wyników dla tych samych seedów oraz wykorzystanie tych samych talii przez porównywanych agentów.

Testami agregacji metryk weryfikuję obliczanie wyniku całkowitego, średniego wy-

niku, wartości postawionych zakładów, odchylenia standardowego, błędu standardowego oraz liczników wyników i akcji. Osobnymi testami pokryłem zapis wyników eksperymentu.

4.10. Podsumowanie

Podsumowując ten rozdział, zdefiniowałem dwa punkty odniesienia dla dalszych eksperymentów. `RandomAgent` reprezentuje strategię pozbawioną wiedzy o wartości kart, natomiast `RuleBasedAgent` wykorzystuje deterministyczny zestaw reguł.

Przygotowałem infrastrukturę, która umożliwia prowadzenie powtarzalnych symulacji, agregowanie wyników i zapisywanie danych poszczególnych epizodów. Ta infrastruktura stanowi fundament do porównywania agentów wykorzystujących metody uczenia przez wzmacnianie z ustalonymi strategiami bazowymi w kolejnych etapach pracy.

5. Metody uczenia przez wzmacnianie

W tym rozdziale przedstawiam elementy uczenia przez wzmacnianie, które bezpośrednio wykorzystuję w eksperymentach. Opisuję w nim w jaki sposób przekształcam obserwacje do reprezentacji numerycznej oraz definiuję sygnał nagrody.

Dalej w rozdziale opisuję najpierw dwa elementy wspólne dla obu zastosowanych algorytmów, czyli sposób kodowania stanu oraz funkcję nagrody, a następnie same algorytmy, mianowicie Deep Q-Network oraz Policy Gradient oparty na metodzie REINFORCE. Rozdział kończę krótkim zestawieniem różnic pomiędzy tymi dwoma podejściami.

5.1. Reprezentacja stanu

Według Macieja Lewczuka, autora artykułu o uczeniu przez wzmacnianie, stan to opis aktualnej sytuacji agenta w środowisku [6].

W moich rozważaniach zastosowałem dwa warianty funkcji stanu, które nazwałem State A i State B. Obie funkcje powstają wyłącznie na podstawie obiektu `UTHObservation`, a więc informacji dostępnych graczowi.

5.1.1. State A: reprezentacja surowa

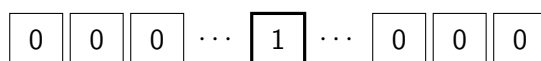
Pierwszy wariant, który określiłem jako State A, stanowi surowy wektor reprezentujący karty. Każda karta jest reprezentowana jako wektor one-hot o długości 52.

Kodowanie jednej karty jako one-hot, a nie jako pojedynczej wartości porządkowej, wynika z tego, co właściwie koduję. Sama ranga karty ma charakter porządkowy i w tej postaci wykorzystuję ją w cechach domenowych State B opisanych w podrozdziale 5.1.2. Tutaj jednak koduję całą kartę, czyli konkretną kombinację rangi i koloru, a kolor nie ma w pokerze żadnej naturalnej hierarchii. Przypisanie 52 kartom pojedynczych, kolejnych numerów mogłoby sugerować sieci nieistniejącą zależność, na przykład że dwie karty o bliskich numerach są sobie w jakimś sensie bliskie. Kodowanie one-hot tego problemu nie ma, ponieważ każda karta dostaje własną, niezależną pozycję w wektorze.

Kodowanie one-hot polega na tym, że każdą wartość ze zbioru możliwych wartości reprezentuję jako wektor samych zer z pojedynczą jedynką na pozycji odpowiadającej tej konkretnej wartości [7]. A zatem każda karta to 52 liczby, z czego 51 to zera. Jedynka stoi na pozycji przypisanej dokładnie tej karcie (np. As pik), jak przedstawiono na rysunku 5.1. Więc agentowi dostarczam informacje o obecności karty. Nieodkryte karty to wektory zerowe.

Do reprezentacji stanu dołączyłem zakodowaną aktualną fazę rozdania oraz maskę sześciu możliwych akcji, dzięki której sieć neuronowa ma bezpośredni dostęp do informacji o tym, które akcje są w danym momencie możliwe do wykonania. Łączny

52 elementy (jedna karta)



jedynka na pozycji
konkretnej karty (np. As pik)

Rys. 5.1. Kodowanie one-hot pojedynczej karty.

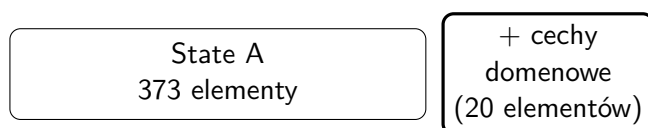
wymiar State A wynosi

$$d_A = 7 \cdot 52 + 3 + 6 = 373. \quad (5.1)$$

Chciałem, aby State A nie zawierał informacji o sile układu pokerowego. Model musi więc samodzielnie nauczyć się korelacji pomiędzy kartami a ich wartością w pokerze.

5.1.2. State B: reprezentacja z cechami domenowymi

Drugi wariant, który nazwałem State B, jest rozszerzeniem surowego wariantu opisanego w podrozdziale 5.1.1 o 20 dodatkowych cech domenowych, tak jak przedstawiłem na rysunku 5.2.



Rys. 5.2. State B jako State A rozszerzone o 20 cech domenowych

Oprócz informacji o kartach, fazie rozdania i dostępnych akcjach, do reprezentacji tego wariantu dołączyłem informacje o właściwościach kart prywatnych, o kategorii najlepszego widocznego układu oraz o strukturze kart, między innymi o parach, zgodności kolorów i możliwości utworzenia strita.

Wymiar drugiego wariantu wynosi

$$d_B = 373 + 20 = 393. \quad (5.2)$$

Porównanie obu reprezentacji pozwala mi ocenić, czy przekazanie modelowi cech pokerowych ułatwia proces uczenia w porównaniu z reprezentacją opartą głównie na surowym kodowaniu kart.

5.2. Funkcja nagrody

Zgodnie z opracowaniem *Google Developers*, funkcja nagrody to liczbowa ocena jakości decyzji, zwracana agentowi przez środowisko po wykonaniu przez niego działania w danym stanie [8].

W obu wariantach zastosowałem nagrodę terminalną, przyznaną dopiero na końcu rozdania. Kroki niekończące rozdania zwracają wartość zero, natomiast po zakończeniu epizodu nagroda wynika bezpośrednio z końcowego rozliczenia finansowego.

Oznacza to, że pojedyncza decyzja podjęta na wczesnym etapie (na przykład przed flopem) nie otrzymuje żadnej bezpośredniej informacji zwrotnej. Algorytm musi zatem rozwiązać problem przypisania zasługi (ang. *credit assignment problem*), czyli ocenić, w jakim stopniu ta wczesna decyzja przyczyniła się do ostatecznego wyniku [9]. Ważną rolę pełni tu współczynnik dyskontowania γ , który omawiam przy opisie obu zastosowanych algorytmów.

Pierwszy wariant, który nazwałem *Reward A*, wykorzystuje zysk netto gracza wyrażony w jednostkach stawki Ante:

$$R_A = \begin{cases} 0, & \text{dla kroku nieterminalnego,} \\ P_{\text{net}}, & \text{dla kroku terminalnego.} \end{cases} \quad (5.3)$$

gdzie P_{net} oznacza zysk netto gracza, czyli sumę wypłat z zakładów Ante, Blind i Play pomniejszoną o wniesione stawki.

Drugi wariant, *Reward B*, to liniowe przeskalowanie wariantu *Reward A* tak, aby standardowa maksymalna nagroda wynosiła jeden:

$$R_B = \frac{R_A}{6}. \quad (5.4)$$

Wartość sześć odpowiada maksymalnej standardowej sumie stawek Ante, Blind i Play w pojedynczym rozdaniu.

Warto podkreślić, że zastosowane przeskalowanie nie jest obcinaniem nagród (ang. *reward clipping*). W przeciwieństwie do clippingu, który sztywno ogranicza wartości nagród do stałego przedziału (np. $[-1, 1]$), liniowe skalowanie zachowuje pełne proporcje sygnału. Dzięki temu przy bardzo wysokich wypłatach z zakładu Blind nagroda R_B nadal może przekroczyć wartość jeden, nie tracąc informacji o skali wygranej [10].

Porównanie obu podejść pozwala mi na zbadanie wpływu samej skali nagrody na stabilność i szybkość procesu uczenia.

5.3. Deep Q-Network

Pierwszym algorytmem uczenia przez wzmacnianie, który zaimplementowałem, jest Deep Q-Network, w skrócie DQN. Jest to jeden z najbardziej znanych algorytmów tego typu, więc zanim przejdę do szczegółów mojej implementacji, chciałbym krótko wyjaśnić, na czym w ogóle polega.

DQN uczy się oceniać, jak dobra jest dana akcja w danym stanie gry, przypisując jej liczbę zwaną wartością Q . Im wyższa wartość Q danej akcji, tym lepszej decyzji ona według agenta odpowiada. W klasycznym podejściu tablicowym taką wartość przecho-

wuje się osobno dla każdej możliwej pary stanu i akcji, w osobnej komórce tabeli. W Ultimate Texas Hold'em samych kombinacji kart widocznych graczowi jest dziesiątki milionów już po flopie, a po odkryciu kolejnych kart wspólnych jeszcze więcej, więc liczba możliwych stanów jest ogromna i taka tabela byłaby niewykonalna. Zamiast niej DQN wykorzystuje sieć neuronową (ang. *neural network*), która nie zapamiętuje wartości Q dla każdego stanu z osobna, lecz uczy się je szacować na podstawie widocznych regularności, zgodnie z tym, co napisałem w rozdziale 2.

5.3.1. Sieć Q i maskowanie akcji

Do przybliżania wartości Q zaimplementowałem sieć QNetwork. Przyjmuje ona na wejściu jeden z dwóch wariantów reprezentacji stanu, State A albo State B, opisanych w podrozdziale 5.1, a na wyjściu zwraca sześć liczb, po jednej dla każdej z sześciu możliwych akcji naraz. Dzięki temu, żeby ocenić wszystkie dostępne w danym momencie decyzje, wystarczy mi jedno przejście sieci, a nie sześć osobnych. W środku umieściłem dwie warstwy ukryte po 256 neuronów z funkcją aktywacji ReLU [11]:

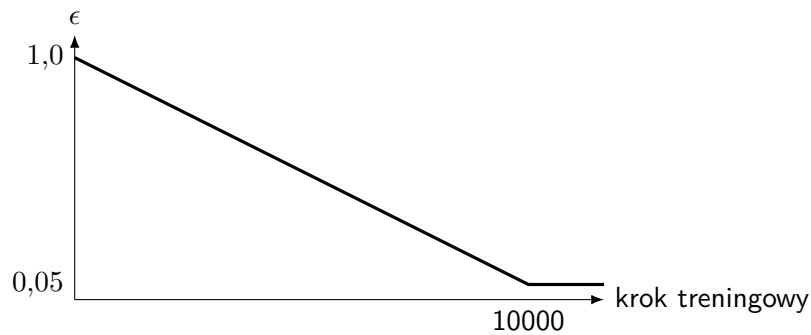
$$d_{\text{state}} \rightarrow 256 \rightarrow 256 \rightarrow 6, \quad d_{\text{state}} \in \{373, 393\}. \quad (5.5)$$

Nie każda z tych sześciu akcji jest jednak legalna w każdej fazie rozdania, na przykład spasowanie jest dostępne dopiero po odkryciu wszystkich kart wspólnych. Zanim agent podejmie decyzję, maskuję więc wartości Q odpowiadające akcjom niedozwolonym w bieżącej fazie, dzięki czemu nigdy nie może on wybrać nielegalnej akcji. To samo maskowanie stosuję również przy wyznaczaniu celu dla następnego stanu, co opisuję w podrozdziale o sieci docelowej i celu Bellmana.

5.3.2. Eksploracja epsilon-greedy

Sieć Q uczy się wyłącznie na podstawie akcji, które agent faktycznie wykonał, więc żeby poznać wartość jakiejś akcji, musi ją najpierw wypróbować. Gdybym pozwolił agentowi zawsze wybierać akcję, którą aktualnie ocenia jako najlepszą, mógłby on utknąć przy pierwszej, niekoniecznie najlepszej ocenie i nigdy nie sprawdzić alternatywy, która mogłaby dać lepszy wynik. Z tego powodu podczas treningu zastosowałem powszechnie stosowaną w uczeniu przez wzmacnianie strategię epsilon-greedy [12].

Strategia ta polega na tym, że z pewnym prawdopodobieństwem, oznaczanym grecką literą epsilon, agent wybiera zupełnie losową legalną akcję, a w pozostałych przypadkach wybiera akcję, którą aktualnie ocenia jako najlepszą. Na początku treningu ustawiam epsilon na 1,0, więc agent na starcie eksploruje w zasadzie losowo, a w miarę postępu treningu wartość epsilon liniowo maleje, aż do 0,05 po 10000 kroków, tak jak przedstawiłem na rysunku 5.3. Agent z czasem więc coraz częściej korzysta z tego, czego się już nauczył, a coraz rzadziej próbuje czegoś nowego.



Rys. 5.3. Liniowy spadek wartości epsilon w trakcie treningu.

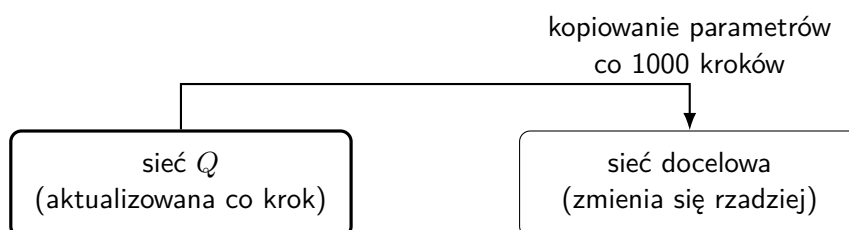
5.3.3. Bufor doświadczeń

Kolejnych przejść pomiędzy stanami nie wykorzystuję do aktualizacji sieci od razu, tylko zapisuję je w buforze doświadczeń, (ang. *experience replay*), zaimplementowanym jako klasa `ReplayBuffer` przechowująca obiekty `Transition` [13].

Gdybym uczył sieć bezpośrednio na kolejnych przejściach z jednego rozdania, tak jak są rozgrywane, kilka aktualizacji z rzędu dotyczyłoby wciąż tych samych dwóch kart gracza i tego samego, stopniowo odkrywanego układu kart wspólnych. Sieć zaczęłaby więc dopasowywać się do wąskiego wycinka gry, zamiast uczyć się zależności ogólnej dla wszystkich możliwych rozdania. Bufor doświadczeń rozwiązuje ten problem. Po pierwsze, przejście trafia najpierw do bufora, a nie od razu do sieci. Po drugie, do każdej aktualizacji losuję minibatch przejść pochodzących z wielu różnych, niepowiązanych ze sobą rozdania, dzięki czemu pojedyncza aktualizacja opiera się na zróżnicowanej próbce, a nie na fragmencie jednego rozdania.

5.3.4. Sieć docelowa i cel Bellmana

Oprócz sieci Q , którą aktualizuję na bieżąco, utrzymuję drugą, osobną sieć o identycznej architekturze, zwaną siecią docelową (ang. *target network*). Jej parametry synchronizuję z siecią aktualizowaną tylko okresowo, a nie na bieżąco. Zrobiłem to, ponieważ bez tego rozróżnienia cel, do którego dąży trening, zmieniałby się przy każdej pojedynczej aktualizacji razem z samą siecią, tak jakbym próbował trafić do celu, który sam co chwilę się przesuwa, co utrudniałoby zbieżność metody. Zależność między tymi dwiema sieciami przedstawiłem na rysunku 5.4.



Rys. 5.4. Sieć Q i sieć docelowa.

Cel dla aktualizacji wyznaczam zgodnie z równaniem Bellmana [14], jednym z najbardziej podstawowych równań w uczeniu przez wzmacnianie. Mówi ono, że wartość podjęcia danej akcji w danym stanie to suma nagrody otrzymanej natychmiast oraz zdyskontowanej wartości najlepszej możliwej akcji w stanie następnym. Dla przejścia nieterminalnego cel Bellmana ma więc postać

$$y_t = r_t + \gamma \max_{a' \in A(s_{t+1})} Q_{\theta^-}(s_{t+1}, a'), \quad (5.6)$$

gdzie $A(s_{t+1})$ to zbiór akcji dozwolonych w stanie s_{t+1} , a θ^- to parametry sieci docelowej.

natomiast dla przejścia terminalnego, czyli takiego, po którym rozdanie się kończy, nie istnieje już żaden stan następny, więc cel ogranicza się do samej otrzymanej nagrody

$$y_t = r_t. \quad (5.7)$$

Współczynnik γ w tym równaniu, zwany współczynnikiem dyskontowania, określa, jak bardzo agent ma cenić nagrody odległe w czasie względem nagrody otrzymanej natychmiast [15], [16]. Wartość bliska zeru sprawiłaby, że agent liczyłby się właściwie tylko z najbliższą nagrodą, natomiast wartość bliska jedynce sprawia, że nagrody z dalszej przyszłości mają dla niego niemal taką samą wagę jak nagroda natychmiastowa. W moich eksperymentach przyjąłem $\gamma = 0,99$, czyli wartość bliską jedynce, ponieważ samo rozdanie może obejmować kilka decyzji, a zależy mi na tym, żeby agent brał pod uwagę cały jego przebieg, a nie tylko najbliższy krok.

Funkcję `compute_bellman_targets()` implementuję zgodnie z powyższymi równaniami, maskując przy tym akcje niedozwolone w stanie s_{t+1} w dokładnie taki sam sposób jak przy wyborze akcji przez agenta.

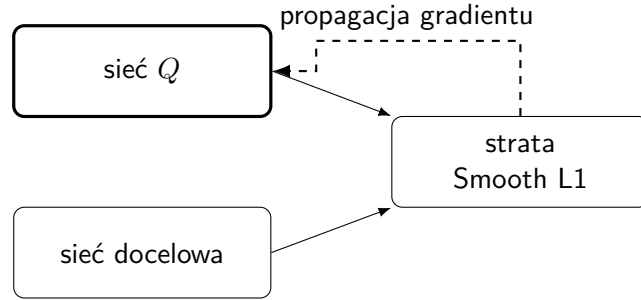
5.3.5. Strata i optymalizacja

Sieć Q uczę poprzez minimalizację funkcji straty (ang. *loss function*), czyli funkcji, która mierzy, jak bardzo bieżąca ocena sieci różni się od wartości oczekiwanej, i którą podczas treningu staram się zminimalizować [17]. W moim przypadku jest to funkcja Smooth L1, liczona pomiędzy $Q_{\theta}(s_t, a_t)$ a wyznaczoną wcześniej wartością docelową y_t , za pomocą klasy `DQN0ptimizer`. W porównaniu ze zwykłym błędem średniokwadratowym Smooth L1 karze duże odchylenia łagodniej, co w moim przypadku ma uzasadnienie, ponieważ jak pokazuje tabela 2.3, pojedyncze rozdanie może zakończyć się wypłatą zakładu Blind w stosunku aż 500:1, a taki rzadki, skrajnie wysoki wynik nie powinien jednorazowo zdominować całej aktualizacji sieci.

Do aktualizacji parametrów na podstawie wyznaczonej straty wykorzystuję optymalizator Adam. Jest to popularny optymalizator, który dla każdego parametru sieci osobno dostosowuje wielkość kroku aktualizacji na podstawie historii jego poprzednich gradientów, dzięki czemu zwykle zbiega szybciej i stabilniej niż podstawowy spadek gra-

dientu ze stałym krokiem [18].

Sieć Q i sieć docelowa uczestniczą więc w każdej aktualizacji razem, ale w różny sposób. Obie wyznaczają wartości potrzebne do policzenia straty, jednak parametry aktualizuje wyłącznie propagacja gradientu przez sieć Q . Parametry sieci docelowej, tak jak opisałem wcześniej, zmienia jedynie okresowe kopiowanie z rysunku 5.4, nigdy propagacja gradientu. Tę różnicę w celu wizualizacji przedstawiłem na rysunku 5.5.



Rys. 5.5. Przepływ gradientu podczas aktualizacji sieci Q .

5.4. Policy Gradient – REINFORCE

Drugim algorytmem uczenia przez wzmacnianie, który zaimplementowałem, jest Policy Gradient oparty na metodzie REINFORCE [5]. W przeciwieństwie do DQN model nie estymuje wartości poszczególnych akcji. Sieć neuronowa bezpośrednio definiuje politykę, czyli rozkład prawdopodobieństwa akcji dla otrzymanego stanu.

5.4.1. Sieć polityki i wybór akcji

Sieć polityki PolicyNetwork przyjmuje wektor utworzony przez wybrany enkoder stanu. Zatem liczba wejść wynosi albo 373 dla State A, albo 393 dla State B. Zastosowałem dwie warstwy ukryte po 256 neuronów z funkcją aktywacji ReLU oraz warstwę wyjściową o sześciu neuronach odpowiadających wszystkim akcjom.

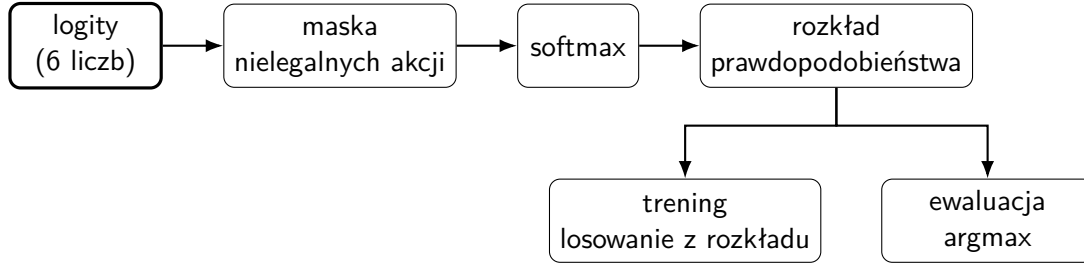
$$d_{\text{state}} \rightarrow 256 \rightarrow 256 \rightarrow 6, \quad d_{\text{state}} \in \{373, 393\}. \quad (5.8)$$

gdzie d_{state} to wymiar wektora stanu, a 6 to liczba akcji.

Sieć zwraca surowe wartości, czyli logity. Przed utworzeniem rozkładu prawdopodobieństwa maskuję akcje niedozwolone w aktualnej fazie rozdania. Dla legalnych akcji prawdopodobieństwa wyznaczam za pomocą funkcji softmax [19].

Zaimplementowałem klasę PolicyGradientAgent, która podczas treningu losuje akcję zgodnie z otrzymanym rozkładem, a nie zawsze wybiera akcję najbardziej prawdopodobną. Dzięki temu agent od czasu do czasu sprawdza również mniej oczywiste decyzje, co pozwala na naturalną eksplorację bez potrzeby dodatkowego mechanizmu losowania, takiego jak epsilon-greedy w DQN. Podczas ewaluacji zależy mi natomiast na jak najlepszej, powtarzalnej decyzji, więc wykorzystuję politykę deterministyczną i

wybieram legalną akcję o największym prawdopodobieństwie, czyli argument maksimum (ang. *argmax*) rozkładu. Cały ten przepływ przedstawiłem na rysunku 5.6.



Rys. 5.6. Wybór akcji w Policy Gradient.

5.4.2. Uczenie metodą REINFORCE

Jedno rozdanie Ultimate Texas Hold'em traktuję jako jeden epizod, a dla każdej decyzji zapisuję zakodowany stan, wybraną akcję, maskę akcji legalnych oraz otrzymaną nagrodę w obiekcie PolicyStep, z których po zakończeniu epizodu składam kompletną Trajectory. Dla każdego kroku wyznaczam wtedy zwrot

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k. \quad (5.9)$$

gdzie G_t to zwrot, czyli suma nagród liczona od kroku t do końca epizodu, T to liczba kroków w epizodzie, r_k to nagroda otrzymana w kroku k , a γ to współczynnik dyskontowania nagród, opisany już przy współczynniku dyskontowania w DQN. Wykładnik $k - t$ rośnie wraz z odległością kroku k od kroku t , więc nagrody odległe w czasie są dyskontowane silniej niż nagrody bliskie.

W eksperymentach przyjąłem $\gamma = 1$, ponieważ optymalizowanym celem jest wynik finansowy całego rozdania, bez preferowania nagrody otrzymanej we wcześniejszym punkcie decyzji.

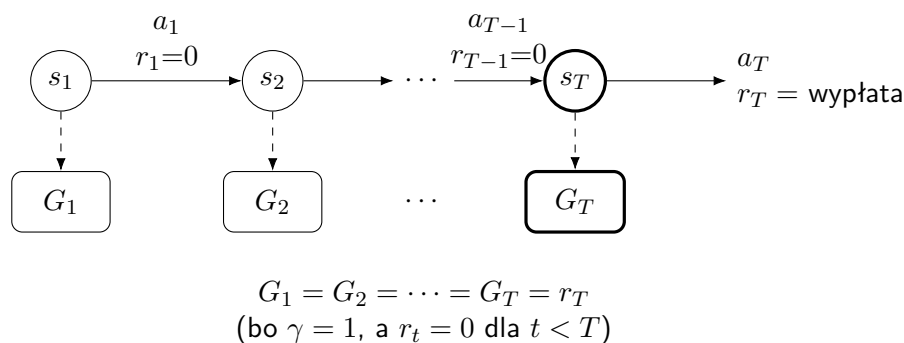
Parametry polityki aktualizuję za pomocą klasy ReinforceOptimizer, zgodnie z klasycznym estymatorem REINFORCE [5]. Dla batcha zawierającego B kompletnych epizodów minimalizowana funkcja straty ma postać

$$L(\theta) = -\frac{1}{B} \sum_{i=1}^B \sum_t G_{i,t} \log \pi_{\theta}(a_{i,t} | s_{i,t}). \quad (5.10)$$

gdzie $G_{i,t}$ to zwrot w kroku t epizodu i , $a_{i,t}$ i $s_{i,t}$ to odpowiednio akcja i stan w tym kroku, a $\pi_{\theta}(a_{i,t} | s_{i,t})$ to prawdopodobieństwo wybrania tej akcji przez aktualną politykę.

Intuicyjnie ta aktualizacja zwiększa prawdopodobieństwo akcji, które doprowadziły do wysokiego zwrotu G_t , oraz zmniejsza prawdopodobieństwo akcji, po których zwrot był niski lub ujemny. Logarytm prawdopodobieństwa wyznacza kierunek zmiany parametrów, natomiast wartość i znak G_t decydują o sile tej zmiany.

Na rysunku s_t oznacza stan w kroku t , a_t podjętą w nim akcję, a r_t otrzymaną



Rys. 5.7. Zwrot G_t w epizodzie REINFORCE.

nagrodę. Ponieważ $r_t = 0$ dla każdego kroku poza ostatnim, a $\gamma = 1$, każdy zwrot G_t sprowadza się do tej samej wartości, czyli końcowej wypłaty rozdania r_T .

Zwroty obliczam niezależnie dla każdego epizodu, a parametry sieci pozostają niezmienione podczas zbierania całego batcha. Dopiero po zebraniu 32 rozdań wykonuję jeden krok optymalizatora Adam. Takie podejście powinno ograniczyć wariancję aktualizacji w porównaniu z wykonywaniem osobnego kroku optymalizacji po każdym rozdaniu.

W domyślnej konfiguracji PolicyGradientConfig zastosowałem learning rate równy 0,001, batch size równy 32 oraz warstwy ukryte (256, 256). Nie stosuję sieci wartości, baseline'u, regularyzacji entropii ani normalizacji zwrotów. Pozostawiam w ten sposób REINFORCE jako względnie prostą metodę Policy Gradient, którą mogę bezpośrednio porównać z DQN.

5.4.3. Reprodukowalność i diagnostyka

Z jednego seeda wyprowadzam osobne, niezależne seedy dla sieci, środowiska i losowania akcji. Dzięki temu mogę później powtórzyć dokładnie ten sam trening, bez obawy, że różne źródła losowości się ze sobą miesza.

Podczas treningu zapisuję między innymi wynik każdego epizodu, liczbę wykonanych kroków, wartość funkcji straty oraz normę gradientu. Dla ustalonego zestawu przykładowych stanów śledzę też, jak zmienia się rozkład prawdopodobieństwa akcji i jego entropia. Zbieram te dane, ponieważ pomagają mi ocenić przebieg uczenia i wychwycić moment, w którym polityka zbyt szybko zawęża się do jednej, wciąż tej samej akcji.

Wyuczoną sieć zapisuję jako checkpoint razem z konfiguracją treningu, wariantem reprezentacji stanu i rodzajem funkcji nagrody, dzięki czemu mogę ją później jednoznacznie odtworzyć i wykorzystać podczas ewaluacji.

5.5. Porównanie podejść

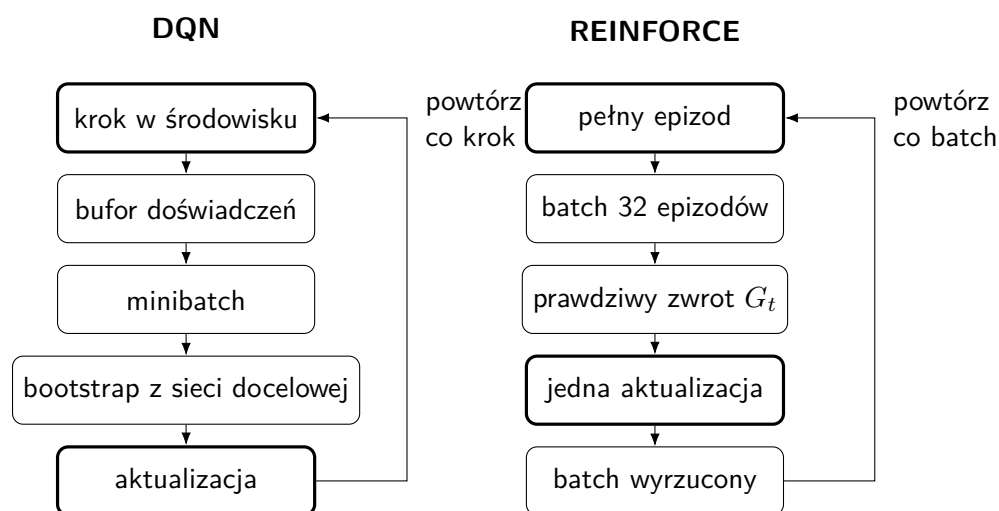
DQN uczy się przybliżonej wartości Q dla każdej pary stanu i akcji, a decyzję podejmuje pośrednio, wybierając akcję o najwyższej ocenionej wartości. Policy Gradient

nie szacuje wartości poszczególnych akcji, tylko uczy się wprost rozkładu prawdopodobieństwa ich wyboru, więc decyzja wynika bezpośrednio z wyjścia sieci polityki.

Różnica ta wpływa też na to, skąd bierze się cel każdej aktualizacji. DQN nie czeka na koniec rozdania, tylko szacuje cel na podstawie wartości ocenionej przez sieć docelową dla następnego stanu. Taką technikę nazywa się bootstrappingiem (ang. *bootstrapping*). Wartość jednego stanu wyznacza się w niej na podstawie oszacowanej wartości stanu następnego, a nie na podstawie faktycznego wyniku całego epizodu [5]. REINFORCE działa odwrotnie czeka, aż rozdanie się zakończy, i aktualizuje politykę na podstawie faktycznie otrzymanej wypłaty, czyli zwrotu G_t z rysunku 5.7.

Ta różnica wpływa również na sposób wykorzystania zebranych danych. Ocena wartości akcji w DQN nie zależy od tego, jaka polityka aktualnie tę akcję wybiera. Estymator REINFORCE zakłada natomiast, że dany epizod pochodzi z polityki właśnie aktualizowanej, dlatego każdy zebrany batch epizodów wykorzystuje tylko do jednej aktualizacji, a potem odrzuca.

Odmienne wygląda też eksploracja. DQN wymaga osobnego mechanizmu epsilon-greedy, ponieważ sama sieć Q zwraca deterministyczną ocenę wartości. W Policy Gradient eksploracja wynika naturalnie ze stochastycznego charakteru wyuczonej polityki. Te różnice przedstawiłem na rysunku 5.8.

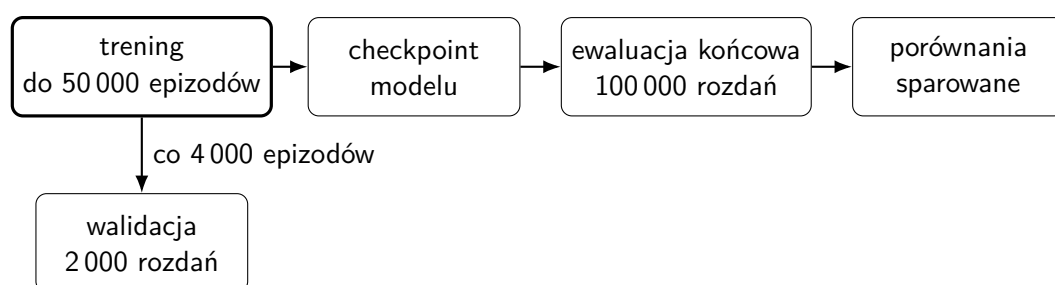


Rys. 5.8. DQN aktualizuje sieć na bieżąco, ponownie wykorzystując stare przejścia z bufora. REINFORCE zbiera cały batch epizodów z aktualnej polityki i wykorzystuje go tylko raz.

6. Metodyka eksperymentów

Przeprowadziłem eksperymenty, w których porównuję wpływ algorytmu uczenia przez wzmacnianie, reprezentacji stanu oraz funkcji nagrody na jakość strategii uzyskiwanej w środowisku Ultimate Texas Hold'em. Wykorzystuję dwa algorytmy opisane w rozdziale 5, tzn. DQN oraz REINFORCE. Za podstawową jednostkę treningu przyjmuję epizod, czyli pojedyncze rozdanie. Polityką nazywam sam wyuczony model decyzyjny, natomiast strategią konkretne zachowanie, które ta polityka realizuje w danym momencie treningu lub ewaluacji.

Właściwości środowiska, sposób generowania rozdań oraz procedura ewaluacji są wspólne dla wszystkich eksperymentów, a poszczególne warianty eksperymentów różnią się wyłącznie badanymi elementami. Cały przebieg jednego eksperymentu, od treningu pojedynczego modelu do porównania wszystkich wariantów, przedstawiłem na rysunku 6.1. Trening trwa do 50 000 epizodów i w jego trakcie co 4 000 epizodów wykonywana jest walidacja na 2 000 rozdaniach, ta część przebiegu ma więc charakter cykliczny. Po zakończeniu treningu wytrenowany model przechodzi jednorazowo do ewaluacji końcowej na 100 000 rozdaniach, a jej wyniki wykorzystuję następnie w porównaniach sparowanych.



Rys. 6.1. Przebieg jednego eksperymentu.

6.1. Macierz eksperymentalna

Tak jak wspomniałem, rozpatruję dwa algorytmy uczenia przez wzmacnianie, dwie reprezentacje stanu oraz dwie funkcje nagrody, co łącznie daje $2 \times 2 \times 2 = 8$ konfiguracji.

Dla każdego algorytmu analizuję reprezentacje RAW oraz FEATURES, odpowiadające State A i State B z podrozdziału 5.1.1 i 5.1.2.

W przypadku funkcji nagrody porównuję bezpośrednią wartość zysku netto z rozdania oraz jej wersję przeskalowaną przez maksymalną wartość stawki.

Wszystkie osiem wariantów przedstawiam w tabeli 6.1.

Tabela 6.1. Konfiguracje wykorzystane w eksperymentach

Algorytm	Reprezentacja	Nagroda
DQN	RAW	zysk netto
DQN	RAW	zysk netto skalowany
DQN	FEATURES	zysk netto
DQN	FEATURES	zysk netto skalowany
REINFORCE	RAW	zysk netto
REINFORCE	RAW	zysk netto skalowany
REINFORCE	FEATURES	zysk netto
REINFORCE	FEATURES	zysk netto skalowany

6.2. Powtarzalność eksperymentów

Ze względu na losowy charakter procesu uczenia pojedynczy przebieg nie jest wystarczający do wiarygodnej oceny konfiguracji. Każdy z ośmiu wariantów trenuję dlatego z użyciem pięciu niezależnych ziaren losowości, co daje w sumie $8 \times 5 = 40$ niezależnych procesów treningowych.

Ten sam zestaw pięciu ziaren wykorzystuję dla wszystkich wariantów. W ten sposób ograniczam wpływ przypadkowych różnic wynikających z losowania rozdań, co również ułatwia porównanie wpływu reprezentacji stanu oraz funkcji nagrody.

6.3. Budżet treningowy

Podstawowy budżet treningowy ustaliłem na 50 000 epizodów dla każdej konfiguracji. Wartość tę wybrałem na podstawie wcześniejszego eksperymentu diagnostycznego, w którym obserwowałem zachowanie wszystkich ośmiu wariantów podczas treningu.

W przypadku DQN zwiększanie liczby epizodów nie prowadziło do regularnego wzrostu jakości polityki. Co ciekawe, w części konfiguracji zaobserwowałem nawet pogorszenie wyniku po dłuższym treningu.

W przypadku REINFORCE część przebiegów osiągała stabilne, proste polityki, takie jak wybieranie tego samego zakładu dla większości stanów. Dodatkowy przebieg diagnostyczny, rozszerzony do 100 000 epizodów, wykazał, że dalsze zwiększanie budżetu nie prowadziło do trwałej poprawy. Polityka przechodziła pomiędzy różnymi strategiami, a następnie ponownie zbliżała się do wcześniej obserwowanego rozwiązania.

Z tego powodu przyjąłem wspólny budżet 50 000 epizodów dla obu algorytmów. Pozwala to również zachować porównywalność kosztu uczenia pomiędzy DQN i REINFORCE.

6.4. Walidacja okresowa

W trakcie treningu wykonuję okresową ewaluację aktualnej polityki. Jej celem jest obserwacja procesu uczenia i budowa krzywych przedstawiających zależność jakości strategii od liczby wykonanych epizodów.

Walidację wykonuję co 4 000 epizodów oraz dodatkowo po zakończeniu 50 000

epizodów. Każdy punkt walidacyjny obejmuje 2 000 rozdań wygenerowanych na podstawie tego samego, ustalonego harmonogramu talii.

Dzięki wspólnemu harmonogramowi kolejne wersje polityki oceniam na identycznym zestawie sytuacji pokerowych. Zmiana wartości oczekiwanej pomiędzy punktami walidacyjnymi wynika więc ze zmiany zachowania agenta, a nie z wykorzystania innego zestawu kart.

Podczas walidacji wyłączam eksplorację. Agent DQN wybiera legalną akcję o najwyższej wartości Q , a agent REINFORCE legalną akcję o najwyższym prawdopodobieństwie zgodnie z aktualną polityką. Pozwala to oceniać deterministyczną strategię wynikającą z aktualnego stanu modelu.

Podkreślam, że harmonogram używany podczas walidacji jest niezależny od rozdań wykorzystywanych w procesie treningowym.

6.5. Rozszerzona analiza treningu

Po wykonaniu podstawowego treningu przeprowadziłem dodatkową analizę wydłużonego treningu. Na podstawie wyników walidacyjnych wybrałem jeden wariant DQN oraz jeden wariant REINFORCE, oba z reprezentacją FEATURES, i wytrenowałem je ponownie do 100 000 epizodów.

Celem tej części bynajmniej nie była próba zastąpienia podstawowego budżetu 50 000 epizodów ani wyszukanie najlepszego chwilowego checkpointu, tylko sprawdzenie, czy dalszy trening prowadzi do trwałej poprawy strategii, jej stabilizacji, czy dalszych oscylacji. Traktuję te wyniki jako analizę uzupełniającą.

6.6. Ewaluacja końcowa

Po zakończeniu treningu oceniam modele na oddzielnym harmonogramie rozdań, niewykorzystywanym ani podczas treningu, ani walidacji, ani nawet podczas wyboru budżetu treningowego.

Kończową ewaluację ustaliłem na 100 000 identycznych rozdań dla każdej ocenianej strategii. Na tym samym zbiorze oceniam również `RandomAgent` oraz `RuleBasedAgent`. Agenci bazowi nie wymagają procesu treningowego, dlatego liczba 100 000 dotyczy w ich przypadku wyłącznie liczby rozegranych rund ewaluacyjnych.

Dla przypomnienia, za podstawową miarą jakości strategii przyjmuję średni zysk netto na jedno rozdanie

$$EV = \frac{1}{N} \sum_{i=1}^N p_i, \quad (6.1)$$

gdzie p_i oznacza zysk netto uzyskany w i -tym rozdaniu, natomiast N oznacza liczbę rozdań ewaluacyjnych.

Oprócz wartości oczekiwanej badam również odchylenie standardowe wyników,

błąd standardowy średniej, średnią wartość postawionych żetonów, liczności wyników rozdania oraz częstości wyboru poszczególnych akcji.

6.7. Porównania sparowane

Wszystkie strategie podczas ewaluacji końcowej rozgrywają ten sam uporządkowany zestaw rozdań, co umożliwia wykorzystanie porównań sparowanych.

Dla dwóch porównywanych strategii A oraz B dla każdego rozdania wyznaczam różnicę

$$d_i = p_{A,i} - p_{B,i}. \quad (6.2)$$

Średnia różnica wynosi

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N d_i. \quad (6.3)$$

Na podstawie rozkładu różnic wyznaczam również odchylenie standardowe, błąd standardowy oraz 95-procentowy przedział ufności. Podejście sparowane ogranicza wpływ zmienności wynikającej z różnych układów kart, ponieważ obie strategie oceniam w tych samych sytuacjach.

7. Wyniki eksperymentów

W tym rozdziale przedstawiam wyniki eksperymentów. Metodykę eksperymentów opisałem w rozdziale 6. Uzyskane wyniki analizuję pod kątem jakości końcowych strategii, wpływu reprezentacji stanu i funkcji nagrody, przebieg procesu uczenia oraz zachowanie modeli po zwiększeniu budżetu treningowego oraz przedstawiam na wykresach w celu lepszej wizualizacji wyników.

Każdą z ośmiu konfiguracji wytrenowałem pięciokrotnie z wykorzystaniem tego samego zestawu pięciu seedów treningowych. Kończącą ocenę każdego modelu przeprowadziłem na wspólnym harmonogramie 100 000 rozdań, który nie był wykorzystywany podczas treningu.

7.1. Wyniki końcowej ewaluacji

Najpierw porównałem końcowe wyniki wszystkich ośmiu podstawowych konfiguracji po 50 000 epizodów treningowych. Dla każdej konfiguracji obliczyłem średnią wartość EV z pięciu niezależnych przebiegów treningowych oraz odchylenie standardowe między seedami. Wyniki przedstawiam w tabeli 7.1.

Tabela 7.1. Wyniki końcowej ewaluacji modeli po 50 000 epizodów treningowych

Algorytm	Reprezentacja	Nagroda	Średnie EV	SD seedów
DQN	RAW	zysk netto	-0,6450	0,0184
DQN	RAW	skalowana	-0,5681	0,0401
DQN	FEATURES	zysk netto	-0,4076	0,0080
DQN	FEATURES	skalowana	-0,4165	0,0169
REINFORCE	RAW	zysk netto	-0,3785	0,0074
REINFORCE	RAW	skalowana	-0,3804	0,0014
REINFORCE	FEATURES	zysk netto	-0,3823	0,0014
REINFORCE	FEATURES	skalowana	-0,3211	0,0657

Najwyższe średnie EV spośród wariantów DQN uzyskała konfiguracja FEATURES z funkcją nagrody opartą na zysku netto. Jej średni wynik wyniósł $-0,4076$. Dla REINFORCE najwyższy średni wynik uzyskała konfiguracja FEATURES ze skalowaną funkcją nagrody, dla której średnie EV wyniosło $-0,3211$.

Widoczna jest wyraźna różnica w zmienności tych wyników. Dla wspomnianego wariantu DQN odchylenie standardowe między pięcioma seedami wyniosło jedynie 0,0080, natomiast dla konfiguracji REINFORCE FEATURES ze skalowaną nagrodą osiągnęło 0,0657. Oznacza to, że wysoka średnia wartość EV wariantu REINFORCE była związana z dużo większym zróżnicowaniem jakości polityk uzyskanych w poszczególnych przebiegach. Do tego zjawiska wracam w dalszej części rozdziału.

Na tym samym harmonogramie 100 000 rozdań ponownie oceniłem dwóch agentów bazowych, Random i RuleBased. Ich wyniki przedstawiam w tabeli 7.2.

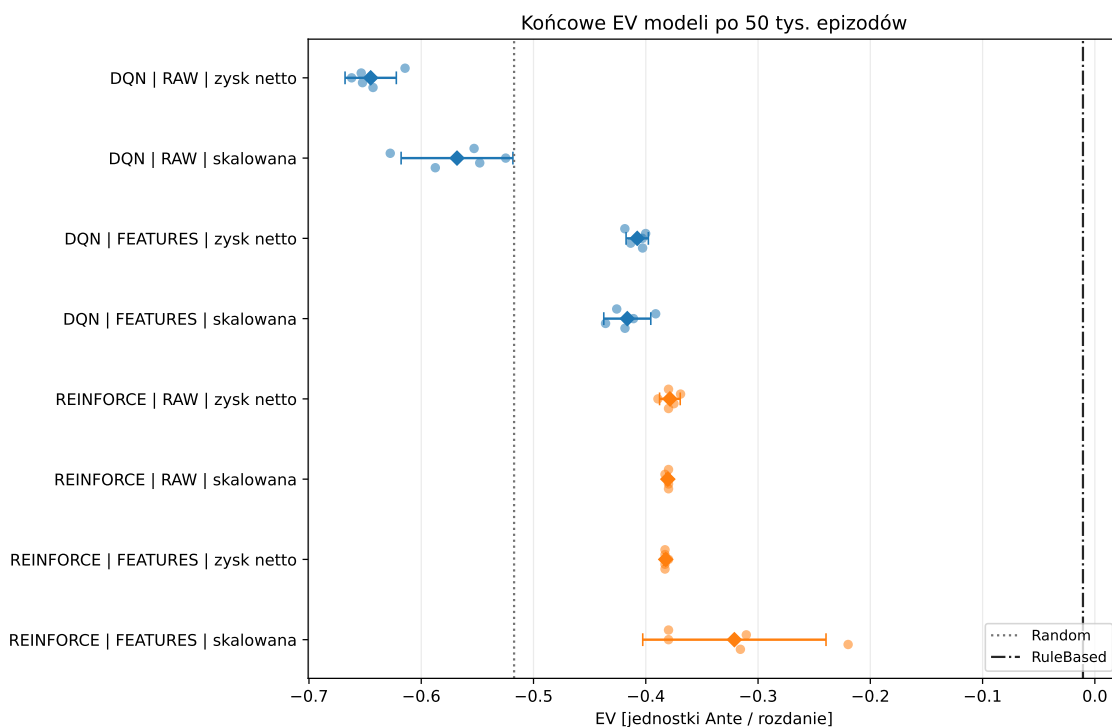
Tabela 7.2. Wyniki końcowej ewaluacji agentów bazowych

Agent	Średnie EV
Random	-0,5173
RuleBased	-0,0106

Wartość uzyskana przez agenta regułowego była znacznie bliżej zera niż wyniki wszystkich wytrenowanych modeli uczenia przez wzmacnianie.

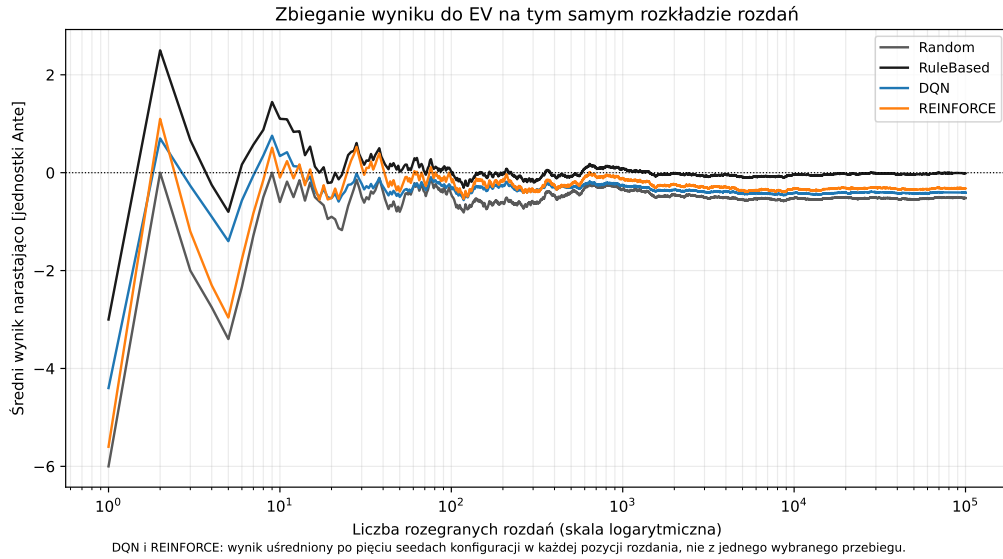
Niemniej jednak większość konfiguracji RL osiągnęła wyższe średnie EV niż agent losowy. Wyjątek stanowiły dwa warianty DQN wykorzystujące reprezentację RAW, których średnie EV wyniosły odpowiednio $-0,6450$ i $-0,5681$. Po zastosowaniu reprezentacji FEATURES oba warianty DQN uzyskały już wynik wyższy niż agent losowy. W przypadku REINFORCE wszystkie cztery badane konfiguracje osiągnęły wyższe średnie EV od agenta losowego.

Zestawienie wyników poszczególnych seedów, ich średnich oraz położenia agentów bazowych przedstawiam na rysunku 7.1. Na osi poziomej znajduje się wartość EV, a na osi pionowej osiem konfiguracji algorytmów. Małe punkty przedstawiają wyniki poszczególnych seedów, większe znaczniki średnie wartości konfiguracji wraz z 95-procentowymi przedziałami ufności, a linie pionowe oznaczają wyniki agentów Random i RuleBased. Widać na nim, że rozrzut punktów wokół średniej jest dla większości konfiguracji REINFORCE zauważalnie większy niż dla DQN, co świadczy o różnicy w powtarzalności obu algorytmów, którą analizuję w dalszej części rozdziału.

**Rys. 7.1.** Końcowe EV ośmiu konfiguracji po 50 000 epizodów treningowych.

Sama informacja o zastosowanym algorytmie nie wystarcza do oceny jakości uzyskanej strategii. Szczególnie w przypadku DQN rezultat zmienia się silnie wraz ze sposobem reprezentacji stanu. W przypadku REINFORCE największą różnicę pomiędzy wariantami obserwuję dla połączenia reprezentacji FEATURES ze skalowaną funkcją nagrody. W kolejnych sekcjach analizuję oba czynniki oddzielnie.

Wszystkie wartości EV opierają się na tym samym, współdzielonym harmonogramie 100 000 rozdań, co pozwala mi prześledzić, jak szybko średni wynik zbiega do swojej ostatecznej wartości. Na rysunku 7.2 przedstawiam ten proces dla agentów Random i RuleBased oraz najlepszych walidacyjnie konfiguracji DQN i REINFORCE. Na osi poziomej, w skali logarytmicznej, znajduje się liczba rozegranych rozdań, a na osi pionowej średni wynik narastająco. Krzywe wszystkich czterech strategii stabilizują się jeszcze przed końcem harmonogramu, co potwierdza, że przyjęta liczba 100 000 rozdań jest wystarczająca do rzetelnego oszacowania EV.



Rys. 7.2. Zbieganie średniego wyniku do EV na współdzielonym harmonogramie rozdań.

7.2. Wpływ reprezentacji stanu

W celu oceny wpływu sposobu reprezentacji stanu porównałem warianty RAW i FEATURES przy zachowaniu tego samego algorytmu, funkcji nagrody oraz seedu treningowego. Dla każdej pary wyznaczyłem różnicę

$$\Delta EV_{\text{state}} = EV_{\text{FEATURES}} - EV_{\text{RAW}}. \quad (7.1)$$

Zgodnie ze wzorem dodatnia wartość różnicy oznacza zatem przewagę reprezentacji FEATURES. Porównanie wykonałem osobno dla obu algorytmów i obu funkcji nagrody. Jednostką porównania jest w tym przypadku seed treningowy, dlatego każda wartość średnia oraz odpowiadający jej przedział ufności wynikają z pięciu sparowanych

różnic.

Wyniki wszystkich czterech porównań przedstawiam w tabeli 7.3.

Tabela 7.3. Wpływ reprezentacji stanu na EV

Algorytm	Nagroda	ΔEV_{state}	95% CI
DQN	zysk netto	0,2374	[0,2066, 0,2682]
DQN	skalowana	0,1517	[0,0863, 0,2170]
REINFORCE	zysk netto	-0,0037	[-0,0144, 0,0069]
REINFORCE	skalowana	0,0593	[-0,0225, 0,1411]

W przypadku DQN zastosowanie reprezentacji FEATURES doprowadziło do wyraźnej poprawy końcowego EV dla obu funkcji nagrody: w obu przypadkach cały przedział ufności leży powyżej zera. Wynika z tego, że w badanej konfiguracji DQN rozszerzenie wektora stanu o cechy opisujące sytuację pokerową poprawiało jakość uzyskiwanej strategii.

Dla REINFORCE wpływ reprezentacji stanu był znacznie mniej jednoznaczny. Przy funkcji nagrody opartej na zysku netto przedział ufności obejmował zero, co oznacza, że obie reprezentacje prowadziły w tym wariancie do bardzo podobnych wyników końcowych. Przy skalowanej funkcji nagrody punktowe oszacowanie wskazuje na wyższe EV reprezentacji FEATURES, jednak przedział ufności jest znacznie szerszy i również obejmuje zero, więc duże zróżnicowanie wyników między seedami nie pozwala mi uznać tego efektu za równie stabilny jak w przypadku DQN.

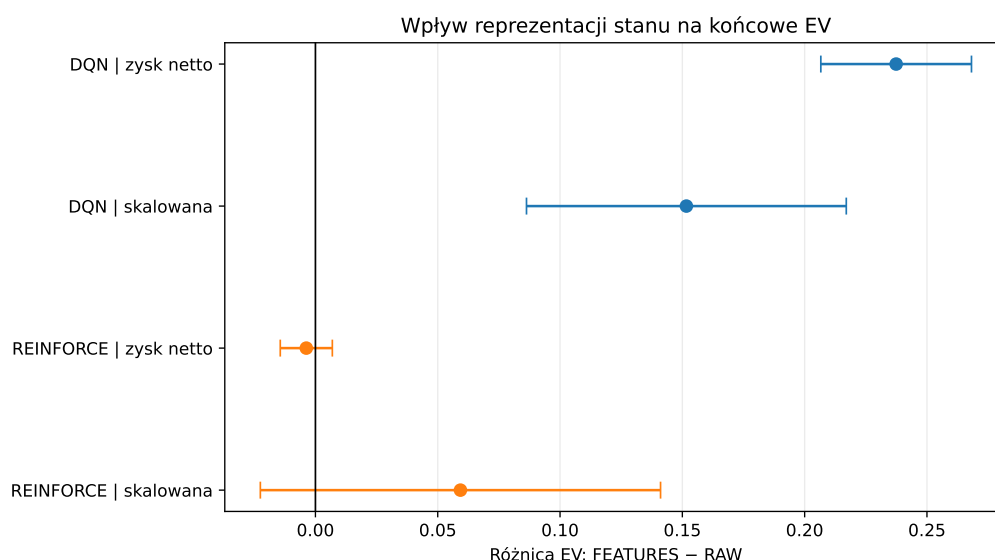
Te same porównania przedstawiam także na rysunku 7.3, gdzie na osi poziomej znajduje się różnica $EV_{\text{FEATURES}} - EV_{\text{RAW}}$, a na osi pionowej cztery kombinacje algorytmu i funkcji nagrody. Punkty przedstawiają średnią różnicę dla pięciu sparowanych seedów treningowych, a wąsy odpowiadają 95-procentowym przedziałom ufności; wartości dodatnie oznaczają przewagę reprezentacji FEATURES. Widać na nim, że przedziały ufności obu wariantów DQN leżą całkowicie po dodatniej stronie zera, natomiast oba przedziały REINFORCE obejmują zero.

Rezultaty wskazują na wyraźnie odmienny wpływ reprezentacji stanu na oba badane algorytmy. DQN korzystał wyraźnie z informacji zawartych w rozszerzonej reprezentacji FEATURES, natomiast dla REINFORCE nie zaobserwowałem równie stabilnego efektu zmiany reprezentacji.

7.3. Wpływ funkcji nagrody

Drugim badanym elementem była funkcja nagrody. Porównałem bezpośredni zysk netto z jego wersją przeskalowaną przy zachowaniu tego samego algorytmu, reprezentacji stanu oraz seedu treningowego. Różnicę zdefiniowałem jako

$$\Delta EV_{\text{reward}} = EV_{\text{net}} - EV_{\text{scaled}}. \quad (7.2)$$



Rys. 7.3. Wpływ reprezentacji stanu na końcowe EV.

Wartość dodatnia oznacza zatem przewagę bezpośredniego zysku netto, natomiast ujemna przewagę nagrody skalowanej. Wyniki wszystkich czterech porównań przedstawiam w tabeli 7.4.

Tabela 7.4. Wpływ funkcji nagrody na EV

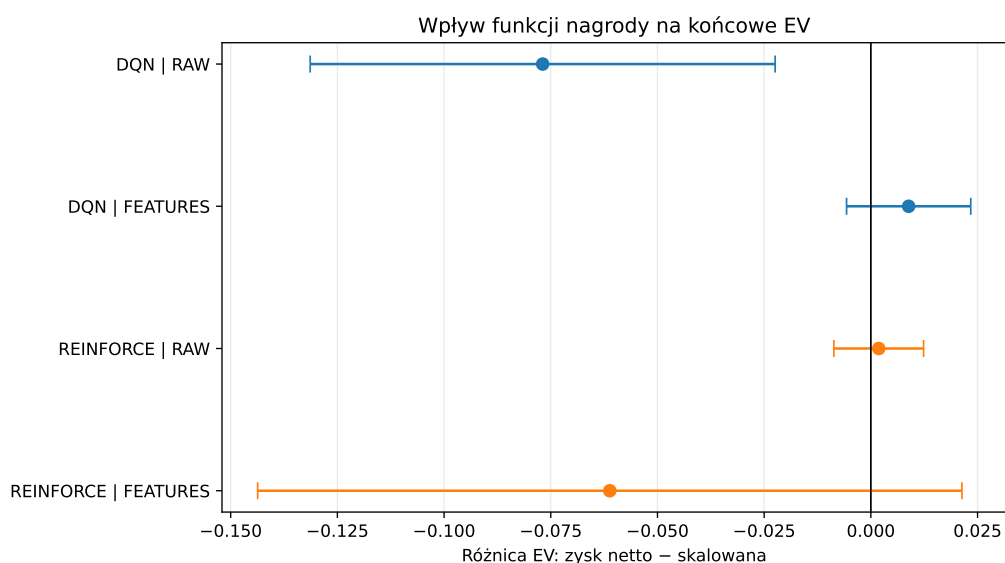
Algorytm	Reprezentacja	$\Delta EV_{\text{reward}}$	95% CI
DQN	RAW	-0,0769	[-0,1314, -0,0224]
DQN	FEATURES	0,0089	[-0,0057, 0,0234]
REINFORCE	RAW	0,0018	[-0,0087, 0,0124]
REINFORCE	FEATURES	-0,0612	[-0,1437, 0,0213]

Dla DQN wykorzystującego reprezentację RAW cały przedział ufności leży poniżej zera. Skalowanie nagrody prowadziło więc w tym wariancie do lepszych wyników końcowych w porównaniu z bezpośrednim wykorzystaniem zysku netto. Sytuacja wyglądała inaczej po zastosowaniu reprezentacji FEATURES: różnica jest niewielka, a przedział obejmuje zero, dlatego na podstawie tych wyników nie można wskazać wyraźnej przewagi jednej z funkcji nagrody dla DQN korzystającego z tej reprezentacji.

Podobny brak jednoznacznej przewagi wystąpił dla REINFORCE z reprezentacją RAW, gdzie przedział ufności również obejmował zero. Największą różnicę punktową dla REINFORCE zaobserwowałem przy reprezentacji FEATURES, gdzie wyższy średni wynik osiągnęła funkcja skalowana. Jednocześnie przedział ufności był tu bardzo szeroki, co jest zgodne z dużym zróżnicowaniem końcowej jakości polityk REINFORCE obserwowanym pomiędzy seedami.

Te same porównania przedstawiam także na rysunku 7.4. Na osi poziomej znajduje się różnica $EV_{\text{net}} - EV_{\text{scaled}}$, a na osi pionowej cztery kombinacje algorytmu i reprezentacji stanu. Punkty przedstawiają średnią różnicę dla pięciu sparowanych seedów

treningowych, a wąsy 95-procentowe przedziały ufności. W przeciwieństwie do wykresu wpływu reprezentacji stanu, tutaj tylko jeden z czterech przedziałów, DQN z reprezentacją RAW, leży całkowicie po jednej stronie zera.



Rys. 7.4. Wpływ funkcji nagrody na końcowe EV.

W przeciwieństwie do reprezentacji stanu nie obserwuję więc uniwersalnego efektu skalowania funkcji nagrody. Jego wpływ zależy od połączenia z algorytmem oraz sposobem reprezentacji stanu. Skalowanie poprawiło wyniki DQN korzystającego z reprezentacji RAW, lecz dla DQN z reprezentacją FEATURES różnica była niewielka. W przypadku REINFORCE punktowo najlepszy wynik uzyskała konfiguracja FEATURES ze skalowaną nagrodą, ale wynik ten był jednocześnie silnie zależny od seedu treningowego.

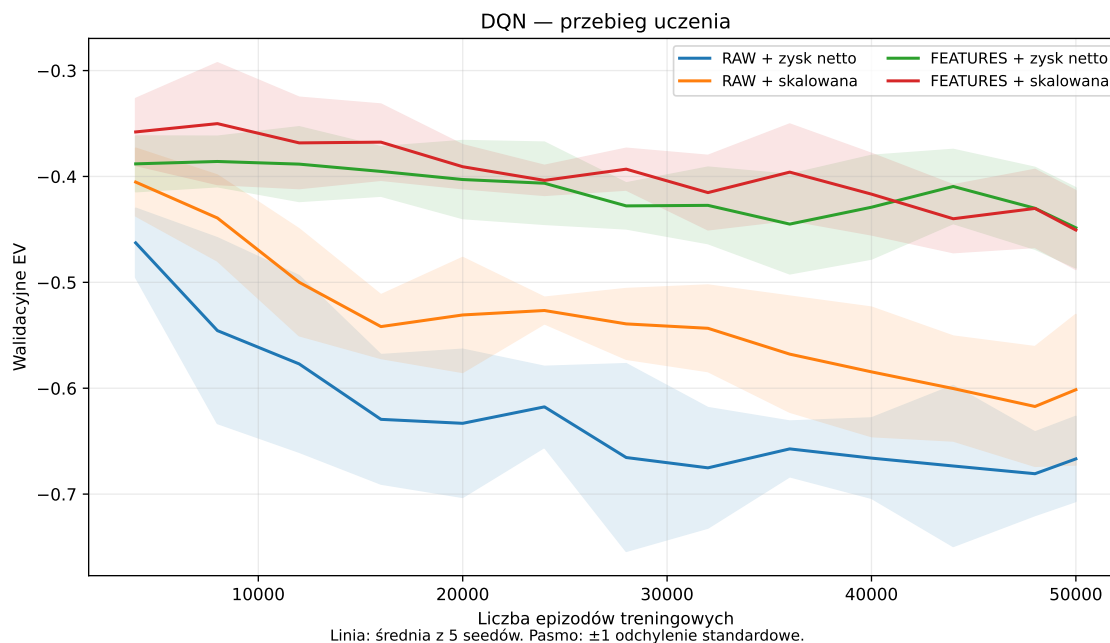
7.4. Przebieg procesu uczenia

Końcowa ewaluacja pozwala porównać jakość wytrenowanych polityk, ale nie pokazuje, w jaki sposób wynik ten kształtował się podczas treningu. W tym celu wykorzystuję okresowe pomiary walidacyjne wykonywane zgodnie z procedurą opisaną w rozdziale 6. Każdy punkt krzywej odpowiada ewaluacji aktualnej polityki na 2 000 wspólnych rozdań walidacyjnych.

Wyniki walidacyjne traktuję jako opis dynamiki procesu uczenia, a nie jako zamiennik końcowej ewaluacji na 100 000 niezależnych rozdań. Mniejsza liczba rozdań w pojedynczym punkcie powoduje większą zmienność oszacowanego EV, jednak wspólny harmonogram walidacyjny umożliwia bezpośrednie porównywanie kolejnych checkpointów.

Na rysunku 7.5 przedstawiam przebieg treningu czterech konfiguracji DQN. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej walidacyjne EV. Linie przedstawiają średnie z pięciu seedów, a pasma jedno odchylenie standardowe między przebiegami treningowymi. Widać na nim, że warianty FEATURES przez cały

trening utrzymują się wyraźnie powyżej wariantów RAW, a rozstęp między krzywymi nie zmniejsza się istotnie z czasem.



Rys. 7.5. Przebieg uczenia czterech konfiguracji DQN.

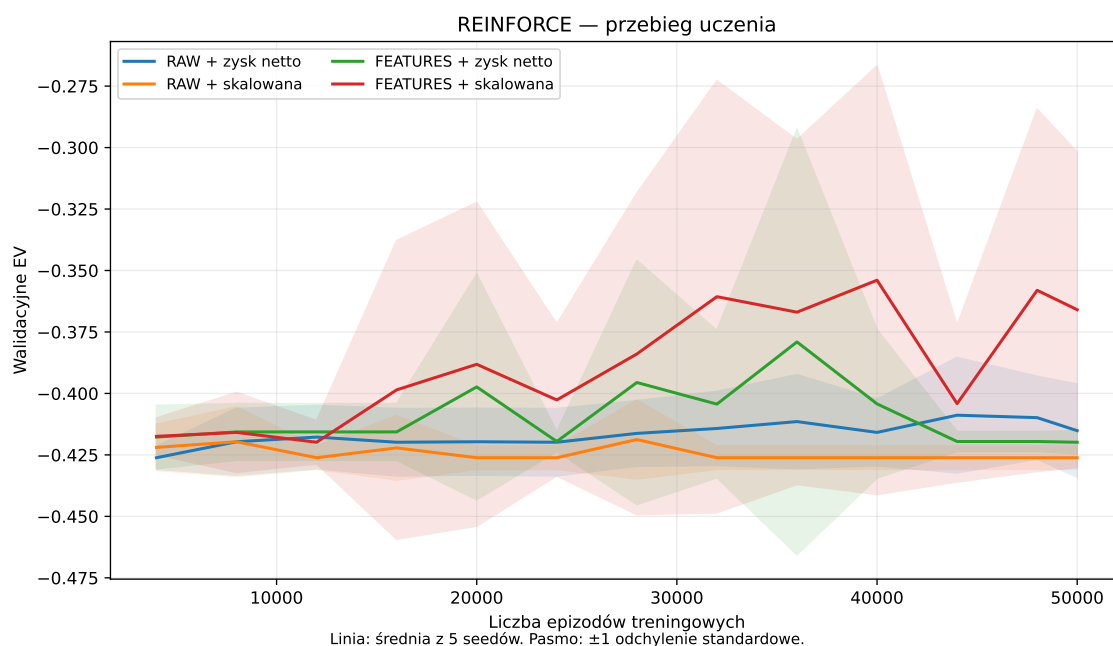
Już w trakcie treningu widoczna jest różnica pomiędzy reprezentacjami RAW i FEATURES. Oba warianty wykorzystujące FEATURES utrzymują przeciętnie wyższe EV niż odpowiadające im warianty RAW. Jest to zgodne z wynikami końcowej ewaluacji przedstawionymi w sekcji 7.2.

Jednocześnie krzywe nie wykazują monotonicznego wzrostu jakości polityki. Po okresach poprawy występują lokalne spadki wartości walidacyjnego EV. Oznacza to, że dalsze wykonywanie aktualizacji sieci nie gwarantuje poprawy strategii na każdym kolejnym checkpointcie. Zjawisko to występuje również dla konfiguracji FEATURES, które ostatecznie uzyskały najwyższe wyniki spośród wariantów DQN.

Analogiczne wyniki dla REINFORCE przedstawiam na rysunku 7.6, z tymi samymi osiami i tym samym sposobem uśredniania. Krzywe REINFORCE są znacznie mniej gładkie niż dla DQN, a pasma odchylenia standardowego są szersze i w wielu punktach przecinają się nawzajem, co wskazuje, że w tym samym momencie treningu różne konfiguracje mogą osiągać porównywalną jakość polityki.

W przypadku REINFORCE zależności pomiędzy konfiguracjami są mniej regularne niż dla DQN. Szczególnie widoczne są okresy zwiększonego rozrzutu pomiędzy seedami, podczas których poszczególne przebiegi tej samej konfiguracji prowadzą do polityk o wyraźnie różnej jakości.

Najbardziej widoczne jest to dla konfiguracji FEATURES ze skalowaną funkcją nagrody. Wariant ten osiągnął najwyższe średnie EV spośród konfiguracji REINFORCE w końcowej ewaluacji, ale jednocześnie charakteryzował się największym odchyleniem stan-



Rys. 7.6. Przebieg uczenia czterech konfiguracji REINFORCE.

dardowym między seedami. Krzywe walidacyjne pokazują, że zróżnicowanie to powstaje już podczas procesu uczenia i nie jest wyłącznie efektem końcowej ewaluacji.

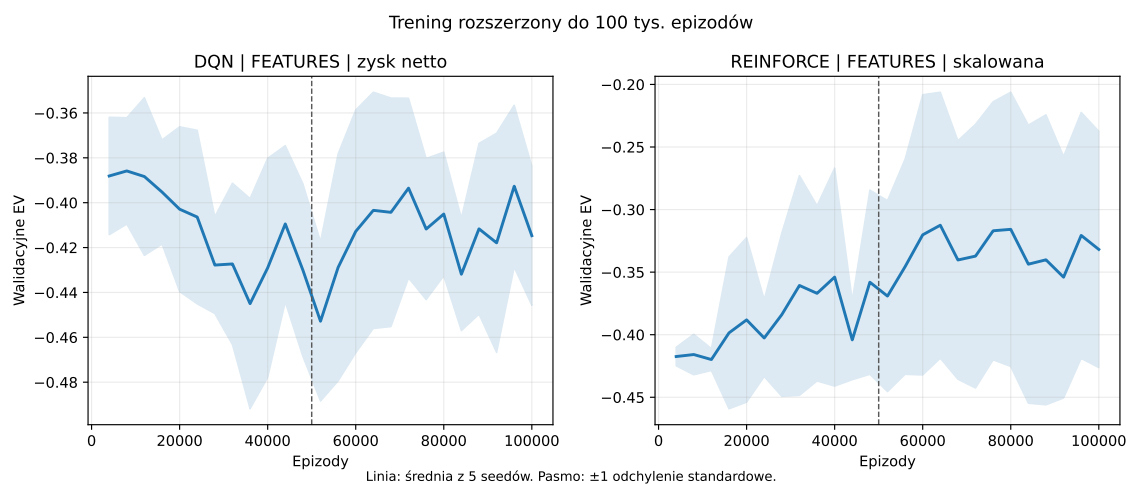
7.5. Wpływ zwiększenia budżetu treningowego

Po zakończeniu podstawowych eksperymentów rozszerzyłem trening dwóch konfiguracji wybranych na podstawie wyników walidacyjnych. Dla DQN była to konfiguracja FEATURES z nagrodą opartą na zysku netto, natomiast dla REINFORCE konfiguracja FEATURES ze skalowaną funkcją nagrody. Wszystkie pięć seedów obu konfiguracji kontynuowałem do 100 000 epizodów.

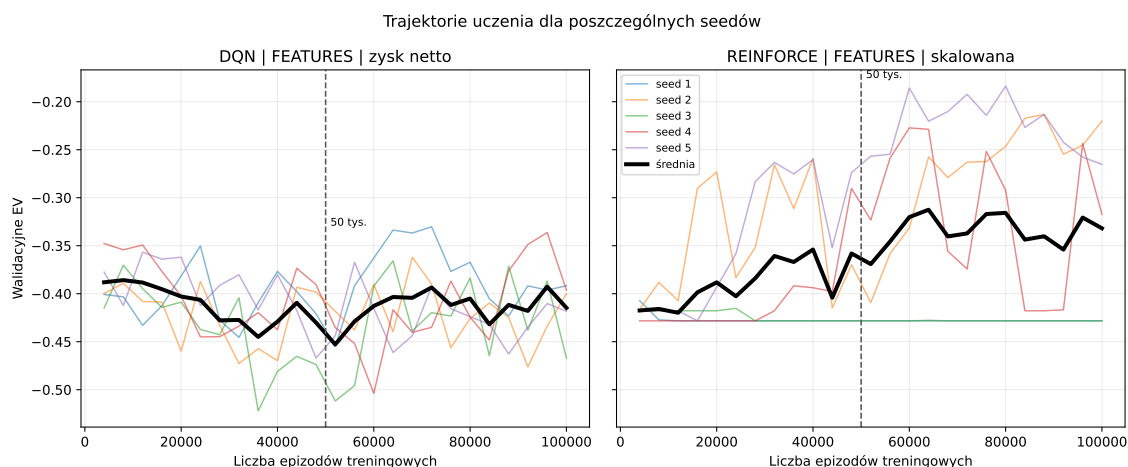
Przebieg walidacyjnego EV uśredniony po pięciu seedach przedstawiam na rysunku 7.7. Na osi poziomej znajduje się liczba epizodów treningowych do 100 000, a na osi pionowej walidacyjne EV; linia pionowa oznacza podstawowy budżet 50 000 epizodów. Dla DQN krzywa po przekroczeniu tej granicy kontynuuje trend wzrostowy widoczny już w podstawowym budżecie, natomiast dla REINFORCE wzrost średniej jest wyraźniejszy dopiero po 50 000 epizodach.

Uśredniona krzywa nie pokazuje jednak, czy poprawa dotyczy wszystkich seedów w podobnym stopniu. Trajektorie poszczególnych przebiegów przedstawiam na rysunku 7.8, z tymi samymi osiami co powyżej, dodatkowo z rozbiciem na pięć kolorów odpowiadających poszczególnym seedom oraz naniesioną krzywą średnią. Dla REINFORCE widoczny jest seed o trajektorii całkowicie płaskiej przez cały trening, co jest zgodne ze zjawiskiem degeneracji polityki do jednej, niezależnej od stanu akcji, które opisuję szerzej w sekcji 7.7.

Dla DQN wydłużenie treningu prowadziło do stosunkowo spójnej poprawy. W



Rys. 7.7. Przebieg walidacyjnego EV wybranych konfiguracji DQN i REINFORCE do 100 000 epizodów.



Rys. 7.8. Trajektorie walidacyjnego EV dla pięciu seedów wybranych konfiguracji DQN i REINFORCE.

końcowej ewaluacji na wspólnych 100 000 rozdaniach średnie EV pięciu modeli wzrosło z $-0,4076$ po 50 000 epizodów do $-0,3859$ po 100 000 epizodów. Średnia sparowana poprawa wyniosła

$$\Delta EV_{100k-50k} = 0,0217, \quad (7.3)$$

a 95-procentowy przedział ufności po pięciu seedach był równy $[0,0022, 0,0412]$.

Co istotne, poprawę końcowego EV zaobserwowałem dla wszystkich pięciu seedów DQN. Nie oznacza to monotonicznej poprawy na każdym etapie treningu, co widać na krzywych walidacyjnych, ale rezultat po 100 000 epizodów był dla każdego seedu lepszy niż wynik odpowiadającego mu modelu po 50 000 epizodów.

Dla REINFORCE średnie EV wzrosło z $-0,3211$ do $-0,2943$. Średnia różnica wyniosła $0,0268$, a więc punktowo była nawet większa niż dla DQN. Jej 95-procentowy przedział ufności był jednak równy $[-0,0362, 0,0898]$ i obejmował zero.

Wynik ten był konsekwencją dużego zróżnicowania zmian pomiędzy poszczególnymi seedami. Niektóre przebiegi REINFORCE poprawiły się znacznie, podczas gdy inne praktycznie nie zmieniły zachowania lub uzyskały nieznacznie gorszy wynik. Wydłużenie treningu poprawiło więc średni rezultat tej konfiguracji, ale nie usunęło problemu niestabilności procesu uczenia.

Wyniki rozszerzenia budżetu podsumowuję w tabeli 7.5.

Tabela 7.5. Wpływ zwiększenia budżetu treningowego z 50 000 do 100 000 epizodów

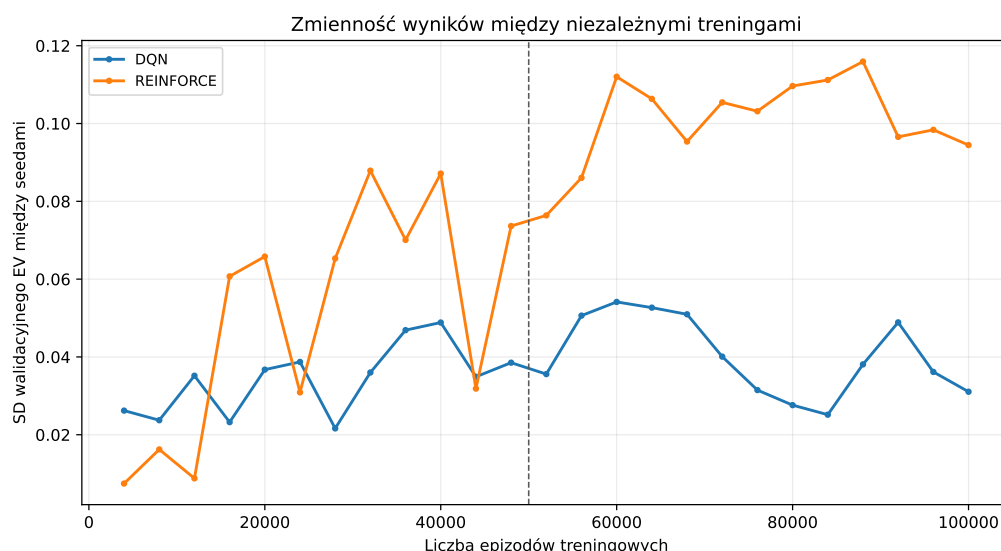
Algorytm	EV 50k	EV 100k	Różnica	95% CI
DQN	-0,4076	-0,3859	0,0217	$[0,0022, 0,0412]$
REINFORCE	-0,3211	-0,2943	0,0268	$[-0,0362, 0,0898]$

Rozszerzony eksperyment należy traktować jako analizę uzupełniającą. Podstawowe porównania reprezentacji stanu i funkcji nagrody pozostają oparte na pełnej macierzy eksperymentalnej przy wspólnym budżecie 50 000 epizodów.

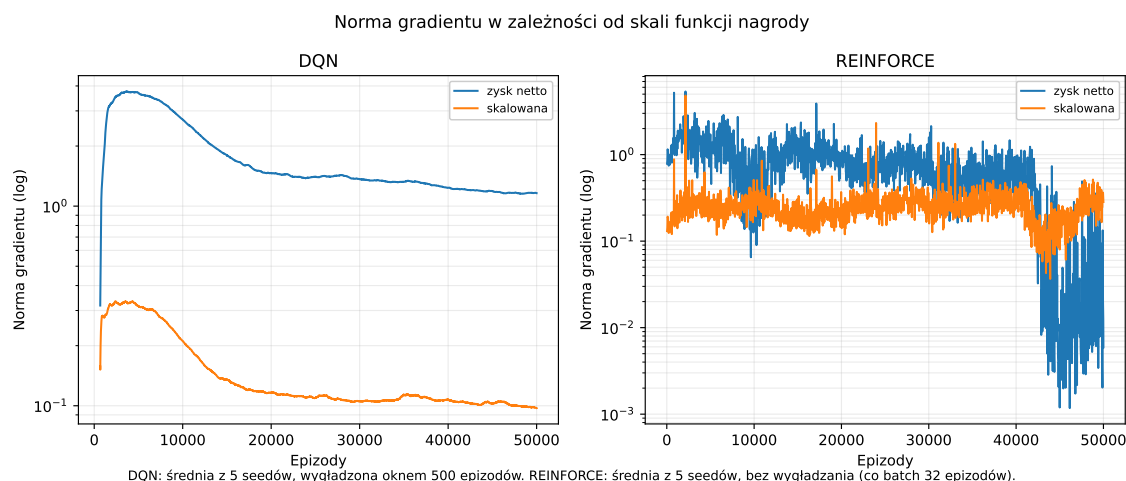
7.6. Stabilność procesu uczenia

Różnice między DQN i REINFORCE są widoczne nie tylko w średnim EV, ale również w powtarzalności procesu treningowego. W celu przedstawienia tego zjawiska obliczyłem dla kolejnych checkpointów odchylenie standardowe walidacyjnego EV pomiędzy pięcioma seedami wybranych konfiguracji.

Wyniki przedstawiam na rysunku 7.9. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej odchylenie standardowe walidacyjnego EV pomiędzy pięcioma seedami wybranych konfiguracji DQN i REINFORCE. Krzywa REINFORCE przez większość treningu leży wyraźnie powyżej krzywej DQN i po 50 000 epizodach nadal rośnie, podczas gdy zmienność DQN pozostaje w przybliżeniu na tym samym poziomie.



Rys. 7.9. Zmienność walidacyjnego EV między seedami podczas treningu.



Rys. 7.10. Norma gradientu w zależności od skali funkcji nagrody.

DQN utrzymywał relatywnie niewielki rozrzut pomiędzy niezależnymi przebiegami. REINFORCE wykazywał natomiast okresy znacznie większej zmienności, co jest zgodne zarówno z krzywymi uczenia, jak i wynikami końcowej ewaluacji. Oznacza to, że w badanym ustawieniu REINFORCE częściej prowadził do jakościowo różnych polityk pomimo wykorzystania tej samej konfiguracji hiperparametrów.

Dodatkową informację o przebiegu optymalizacji daje analiza normy gradientu. Na rysunku 7.10 porównuję jej przebieg dla obu wariantów funkcji nagrody, dla konfiguracji wykorzystujących reprezentację FEATURES. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej, w skali logarytmicznej, norma gradientu uśredniona po pięciu seedach; dla DQN dodatkowo wygładzona oknem 500 epizodów ze względu na aktualizację co krok środowiska. Norma gradientu dla nagrody skalowanej jest w obu algorytmach systematycznie niższa niż dla zysku netto, choć różnica skali jest znacznie większa dla DQN.

Skalowanie nagrody wyraźnie zmieniało skalę gradientów powstających podczas optymalizacji. Nie oznacza to jednak automatycznie poprawy końcowej jakości polityki. Jak pokazały wyniki z sekcji 7.3, mniejsza skala sygnału optymalizacyjnego nie przekładała się na uniwersalną przewagę skalowanej funkcji nagrody.

Analiza gradientów stanowi zatem uzupełnienie wyników EV. Pokazuje, że zmiana funkcji nagrody wpływa na przebieg aktualizacji parametrów, ale sama kontrola skali gradientu nie rozwiązuje problemów związanych ze stabilnością uczenia ani nie gwarantuje uzyskania lepszej strategii.

7.7. Analiza zachowania wyuczonych polityk

Wartość EV opisuje końcową jakość strategii, ale nie pokazuje bezpośrednio, jakie zachowanie zostało wyuczone przez agenta. Z tego względu uzupełniłem analizę ilościową o analizę decyzji podejmowanych przez wybrane konfiguracje DQN i REINFORCE.

W trakcie tej analizy zaobserwowałem u części modeli REINFORCE zjawisko nazywane w literaturze dotyczącej metod policy gradient kolapsem polityki (ang. *policy collapse*) albo przedwczesną zbieżnością do rozwiązania deterministycznego. Polega ono na tym, że polityka przestaje zależeć od obserwowanego stanu i niezależnie od niego wybiera zawsze tę samą akcję. Mni i współautorzy zauważają, że tego rodzaju przedwczesna zbieżność do suboptymalnych, deterministycznych rozwiązań jest częstym problemem metod policy gradient, a jednym ze sposobów jej ograniczenia jest dodanie regularyzacji entropii polityki do funkcji celu [20]. Jak opisałem w podrozdziale 5.4, zastosowana w tej pracy konfiguracja REINFORCE nie wykorzystuje ani regularyzacji entropii, ani baseline'u, co zwiększa podatność tej metody na opisywane zjawisko.

Aby ocenić skalę tego zjawiska, dla każdej z ośmiu podstawowych konfiguracji sprawdziłem, dla ilu z pięciu seedów wybierana akcja przed flopem była identyczna we wszystkich 100 000 rozdaniach końcowej ewaluacji, niezależnie od obserwowanego stanu. Dla żadnej z czterech konfiguracji DQN nie zaobserwowałem takiej degeneracji. Dla REINFORCE zjawisko to okazało się natomiast dominujące, co przedstawiam w tabeli 7.6.

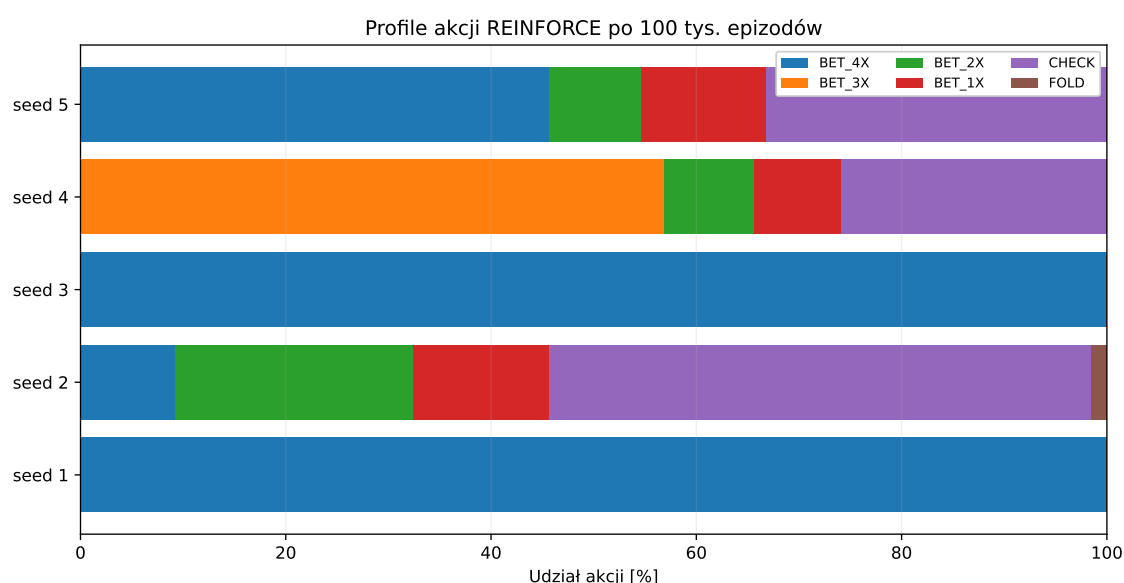
Tabela 7.6. Liczba seedów REINFORCE z polityką zdegenerowaną do jednej akcji przed flopem, spośród pięciu, po 50 000 epizodów

Reprezentacja	Nagroda	Zdegenerowane seedy
RAW	zysk netto	3 / 5
RAW	skalowana	5 / 5
FEATURES	zysk netto	5 / 5
FEATURES	skalowana	2 / 5

Na osiemnaście spośród dwudziestu przebiegów REINFORCE degeneracja do jednej, niezależnej od stanu akcji nie jest więc wyjątkiem dotyczącym pojedynczej konfiguracji, lecz najczęstszym wynikiem całego eksperymentu. Co istotne, konfiguracja REINFORCE FEATURES ze skalowaną funkcją nagrody, która osiągnęła najwyższe średnie

EV spośród wariantów REINFORCE, jest jednocześnie tą, w której degeneracja wystąpiła najrzadziej. Dalszą analizę zachowania prowadzę właśnie na tej konfiguracji, ponieważ zawiera ona jednocześnie modele zdegenerowane i zróżnicowane, co pozwala porównać obie grupy w ramach tego samego zestawu seedów treningowych.

Na rysunku 7.11 przedstawiam rozkład akcji pięciu modeli tej konfiguracji po rozszerzeniu treningu do 100 000 epizodów. Na osi poziomej znajduje się udział poszczególnych akcji w końcowej ewaluacji, a na osi pionowej pięć seedów; każdy pasek przedstawia pełny rozkład akcji jednego modelu. Trzy z pięciu pasków składają się z wielu kolorów i faz rozdania, natomiast dwa są jednokolorowe na całej długości.



Rys. 7.11. Profile akcji pięciu modeli REINFORCE FEATURES ze skalowaną funkcją nagrody.

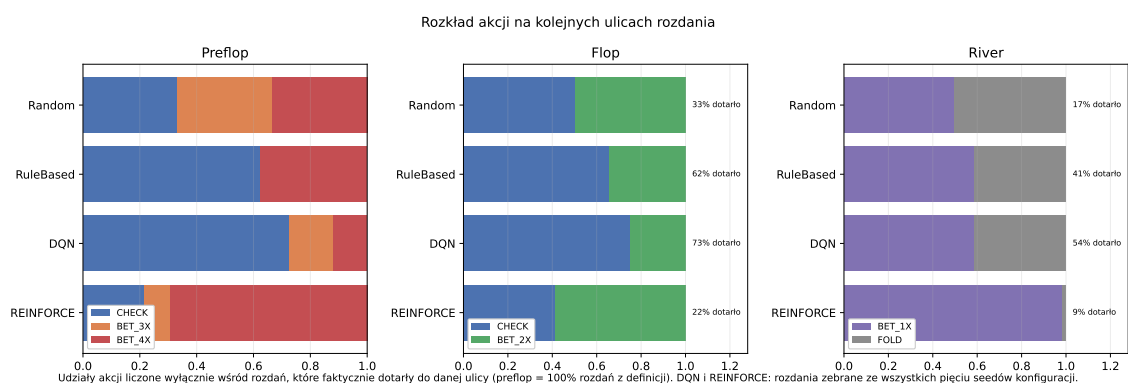
Profile poszczególnych seedów różnią się wyraźnie. Dwa z pięciu modeli wybierały akcję BET_4X we wszystkich 100 000 rozdaniach końcowej ewaluacji. Oznacza to, że polityka tych modeli uległa degeneracji do bardzo prostej strategii, w której decyzja przed flopem przestała zależeć od obserwowanego stanu gry.

Pozostałe przebiegi prowadziły do bardziej zróżnicowanych polityk, które podejmowały również decyzje zależne od kolejnych faz rozdania. Wyjaśnia to część wysokiej zmienności wyników REINFORCE pomiędzy seedami. Ta sama konfiguracja treningowa może prowadzić zarówno do polityki wykorzystującej informacje ze stanu, jak i do rozwiązania zdominowanego przez jedną akcję.

Co istotne, zwiększenie budżetu treningowego do 100 000 epizodów nie usunęło tego zjawiska. Dwa seedy pozostające w prostej polityce po 50 000 epizodów nie zmieniły jej również podczas dalszego treningu. Oznacza to, że sam wzrost liczby epizodów nie wystarczył do wyprowadzenia tych przebiegów z osiągniętego rozwiązania.

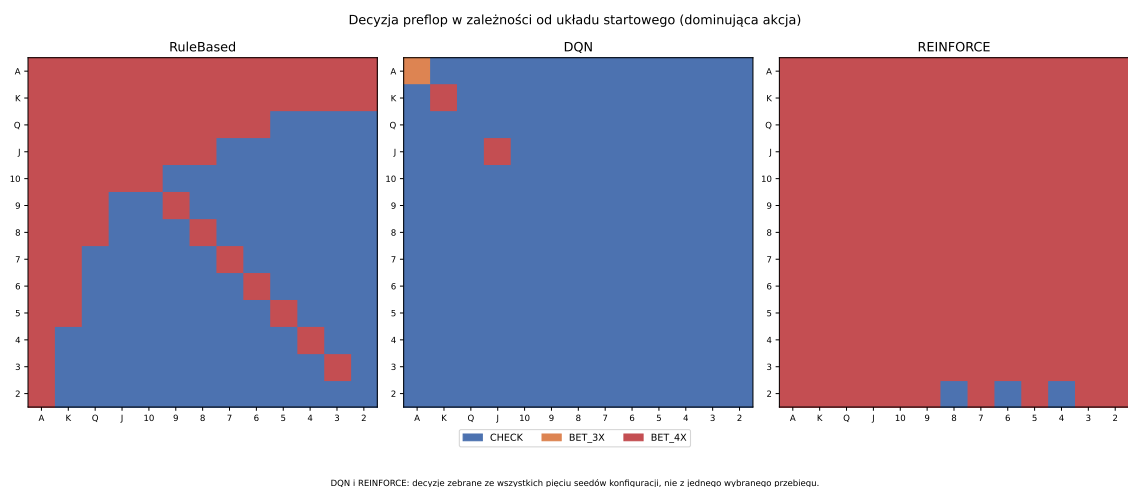
Drugim sposobem analizy zachowania jest porównanie rozkładu akcji na kolejnych ulicach rozdania. Na rysunku 7.12 przedstawiam ten rozkład dla agentów Random i

RuleBased oraz najlepszych walidacyjnie konfiguracji DQN i REINFORCE. Trzy panele odpowiadają preflopowi, flopowi i riverowi; oś pozioma w każdym z nich przedstawia udział poszczególnych akcji wśród rozdań, które faktycznie dotarły do danej ulicy, a przy panelach flopu i riveru dodatkowo podano, jaki odsetek wszystkich rozdań do nich dotarł. REINFORCE dociera do riveru w zaledwie 9% rozdań, podczas gdy dla DQN odsetek ten wynosi 54%, co jest bezpośrednią konsekwencją częstego wyboru najwyższego zakładu już przed flopem.



Rys. 7.12. Rozkład akcji na kolejnych ulicach rozdania.

Trzecim sposobem analizy zachowania jest porównanie decyzji przed flopem dla poszczególnych układów początkowych. Na rysunku 7.13 przedstawiam dominujące decyzje agenta regułowego oraz wybranych walidacyjnie konfiguracji DQN i REINFORCE. Trzy panele odpowiadają trzem strategiom, a układ trzynastu na trzynastcie pól w każdym z nich odpowiada wszystkim kombinacjom dwóch kart początkowych, uporządkowanym według rangi; kolor pola oznacza dominującą akcję wybraną dla danego układu. Agent regułowy tworzy wyraźny, przekątny podział pól, natomiast u konfiguracji RL taki podział jest znacznie słabiej widoczny albo nie występuje wcale.



Rys. 7.13. Dominujące decyzje przed flopem w zależności od układu dwóch kart początkowych.

Agent regułowy posiada wyraźnie zdefiniowany obszar układów początkowych,

dla których wykonuje zakład BET_4X, zgodny z regułami strategii opisanymi w rozdziale 4.4. Nauczone polityki tworzą inne granice decyzyjne, wynikające wyłącznie z procesu optymalizacji oczekiwanej nagrody.

Porównanie pokazuje, że modele RL nie odtworzyły wprost strategii agenta regułowego. Jednocześnie ich zachowanie nie jest równoważne strategii losowej. Szczególnie dla lepszych konfiguracji widoczne są powtarzalne zależności pomiędzy układem początkowym i wybieraną akcją. Oznacza to, że w trakcie treningu modele nauczyły się wykorzystywać część informacji zawartej w obserwowanym stanie, mimo że końcowa jakość strategii pozostawała niższa od jakości heurystyki RuleBased.

7.8. Dyskusja wyników

Przeprowadzone eksperymenty pokazują, że na jakość strategii w Ultimate Texas Hold'em wpływa nie tylko wybór algorytmu uczenia, ale również sposób przedstawienia stanu i właściwości procesu optymalizacji.

Najbardziej jednoznaczny efekt zaobserwowałem dla reprezentacji stanu w DQN. Rozszerzenie RAW o cechy pokerowe w reprezentacji FEATURES poprawiło wynik dla obu funkcji nagrody i efekt ten wystąpił konsekwentnie dla pięciu sparowanych seedów treningowych. Wynik ten wskazuje, że w badanym budżecie treningowym jawne dostarczenie informacji o strukturze sytuacji pokerowej ułatwia DQN wykorzystanie dostępnej obserwacji.

Dla REINFORCE analogicznego, stabilnego efektu reprezentacji stanu nie zaobserwowałem. Najwyższe średnie EV uzyskało połączenie FEATURES ze skalowaną nagrodą, jednak było ono jednocześnie konfiguracją o największej zmienności pomiędzy seedami. Wynik średni nie odzwierciedla więc w tym przypadku typowego rezultatu pojedynczego treningu równie dobrze jak dla DQN.

Wpływ skalowania nagrody również nie był uniwersalny. Dla DQN korzystającego z reprezentacji RAW nagroda skalowana prowadziła do wyższego EV, natomiast po zastosowaniu FEATURES różnica pomiędzy funkcjami nagrody była niewielka. Diagnostyka norm gradientu pokazała, że zmiana skali nagrody wyraźnie wpływała na obserwowaną wielkość sygnału optymalizacyjnego, ale samo ograniczenie tej skali nie gwarantuje poprawy końcowej polityki.

Porównując oba algorytmy przy podstawowym budżecie 50 000 epizodów, REINFORCE osiągnął wyższe średnie EV od DQN dla każdego odpowiadającego sobie połączenia reprezentacji stanu i funkcji nagrody. Nie oznacza to jednak jednoznacznej przewagi REINFORCE. Wyniki tego algorytmu były bardziej zależne od seedu treningowego, a większość przebiegów, osiemnaście z dwudziestu, prowadziła do degeneracji polityki do jednej akcji niezależnej od stanu gry.

DQN osiągał niższe średnie EV, lecz proces treningowy był bardziej powtarzalny. Również eksperyment z większym budżetem treningowym wskazał na różnicę pomiędzy

algorytmami. Wszystkie pięć modeli DQN poprawiło wynik po zwiększeniu budżetu do 100 000 epizodów, natomiast dla REINFORCE poprawa była silnie zależna od konkretnego seedu.

Żadna z badanych konfiguracji RL nie osiągnęła jakości strategii RuleBased. Agent regułowy uzyskał w końcowej ewaluacji EV równe $-0,0106$, podczas gdy najlepsze średnie wyniki podstawowych konfiguracji RL pozostawały wyraźnie ujemne. Pokazuje to, że przy zastosowanych architekturach, hiperparametrach i budżecie treningowym prosta strategia wykorzystująca wiedzę dziedzinową pozostaje znacznie skuteczniejsza.

Nie oznacza to jednak, że modele RL nie nauczyły się użytecznej struktury problemu. Większość konfiguracji osiągnęła wynik wyższy od agenta losowego, a analiza decyzji pokazała zależności pomiędzy obserwowanym stanem i podejmowanymi akcjami. Wyniki sugerują więc, że badane metody były zdolne do uczenia się strategii, lecz jakość tej strategii pozostawała ograniczona.

Na interpretację wyników wpływają również ograniczenia przyjętego zakresu eksperymentalnego. Dla obu algorytmów wykorzystałem jedną ustaloną architekturę i jeden zestaw hiperparametrów. REINFORCE pozostał metodą bez funkcji bazowej, krytyka i dodatkowej regularyzacji entropii. Reprezentacja FEATURES zawiera natomiast ręcznie zaprojektowane cechy pokerowe, dlatego jej przewaga nad RAW pokazuje wartość tej konkretnej informacji dziedzinowej, a nie uniwersalną przewagę każdej reprezentacji o większej liczbie cech.

Z tego względu wyników nie traktuję jako rozstrzygnięcia o ogólnej wyższości jednego algorytmu nad drugim. Pokazują one natomiast wyraźnie, że w badanym środowisku DQN i REINFORCE reagują inaczej na reprezentację stanu, skalę nagrody i zwiększenie budżetu treningowego. Różnica pomiędzy średnią jakością polityki a stabilnością jej uzyskiwania stanowi jeden z najważniejszych rezultatów przeprowadzonych eksperymentów.

Spis rysunków

3.1	Architektura i przepływ informacji w środowisku UTH	17
3.2	Maszyna stanów pojedynczego rozdania UTH	25
5.1	Kodowanie one-hot pojedynczej karty.	42
5.2	State B jako State A rozszerzone o 20 cech domenowych	42
5.3	Liniowy spadek wartości epsilon w trakcie treningu.	45
5.4	Sieć Q i sieć docelowa.	45
5.5	Przepływ gradientu podczas aktualizacji sieci Q	47
5.6	Wybór akcji w Policy Gradient.	48
5.7	Zwrot G_t w epizodzie REINFORCE.	49
5.8	DQN aktualizuje sieć na bieżąco, ponownie wykorzystując stare przejścia z bufora. REINFORCE zbiera cały batch epizodów z aktualnej polityki i wykorzystuje go tylko raz.	50
6.1	Przebieg jednego eksperymentu.	51
7.1	Końcowe EV ośmiu konfiguracji po 50 000 epizodów treningowych.	56
7.2	Zbieganie średniego wyniku do EV na współdzielonym harmonogramie rozdań.	57
7.3	Wpływ reprezentacji stanu na końcowe EV.	59
7.4	Wpływ funkcji nagrody na końcowe EV.	60
7.5	Przebieg uczenia czterech konfiguracji DQN.	61
7.6	Przebieg uczenia czterech konfiguracji REINFORCE.	62
7.7	Przebieg walidacyjnego EV wybranych konfiguracji DQN i REINFORCE do 100 000 epizodów.	63
7.8	Trajektorie walidacyjnego EV dla pięciu seedów wybranych konfiguracji DQN i REINFORCE.	63
7.9	Zmienność walidacyjnego EV między seedami podczas treningu.	65
7.10	Norma gradientu w zależności od skali funkcji nagrody.	65
7.11	Profile akcji pięciu modeli REINFORCE FEATURES ze skalowaną funkcją nagrody.	67
7.12	Rozkład akcji na kolejnych ulicach rozdania.	68

7.13 Dominujące decyzje przed flopem w zależności od układu dwóch kart początkowych.	68
--	----

Spis tabel

2.1	Ranking układów pokerowych od najsilniejszego do naj słabszego	10
2.2	Dostępne decyzje gracza w Ultimate Texas Hold'em	11
2.3	Tabela wypłat zakładu Blind przyjęta w modelu środowiska	11
2.4	Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych . .	12
3.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	17
3.2	Liczba kart przechowywanych w stanie w poszczególnych fazach	22
3.3	Rozliczenie zakładów po showdownie	27
3.4	Tabela wypłat zakładu Blind	28
3.5	Publiczny interfejs klasy UTHGame	30
4.1	Układy początkowe uprawniające do zakładu Play 4× przed flopem . . .	36
4.2	Wyniki agentów bazowych dla 10 000 rozdań	39
4.3	Rozkład wyników agentów bazowych dla 10 000 rozdań	39
6.1	Konfiguracje wykorzystane w eksperymentach	52
7.1	Wyniki końcowej ewaluacji modeli po 50 000 epizodów treningowych . . .	55
7.2	Wyniki końcowej ewaluacji agentów bazowych	56
7.3	Wpływ reprezentacji stanu na EV	58
7.4	Wpływ funkcji nagrody na EV	59
7.5	Wpływ zwiększenia budżetu treningowego z 50 000 do 100 000 epizodów	64
7.6	Liczba seedów REINFORCE z polityką zdegenerowaną do jednej akcji przed flopem, spośród pięciu, po 50 000 epizodów	66

Bibliografia

- [1] T. R. Casino, *Ultimate Texas Hold'em Game Rules*, https://www.everythingtulalip.com/wp-content/uploads/2024/04/ONEclub_0920_GameRules_RackCard_UltimateTexasHoldem-WEB.pdf, Dostęp: 2026-09-01.
- [2] W. of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-09-01.
- [3] J. B. Maverick, *Understanding the House Edge: How Casinos Ensure Profit*, <https://www.investopedia.com/articles/personal-finance/110415/why-does-house-always-win-look-casino-profitability.asp>, Dostęp: 2026-09-01.
- [4] W. of Odds, *What is the House Edge?* <https://wizardofodds.com/gambling/house-edge/>, Dostęp: 2026-09-01.
- [5] R. S. Sutton i A. G. Barto, *Reinforcement Learning: An Introduction*, 2 wyd. Cambridge, Massachusetts: The MIT Press, 2018, Dostęp: 2026-09-01.
- [6] M. Lewczuk, „Jak działa reinforcement learning. Wprowadzenie dla początkujących,” 2024, Dostęp: 2026-09-01.
- [7] dr hab. inż. Jurand Bień, „Uczenie maszynowe: przygotowanie danych – kodowanie zmiennych kategorialnych,” 2022, Dostęp: 2026-09-01.
- [8] G. for Developers, „Słowniczek systemów uczących się,” 2026, Dostęp: 2026-09-01.
- [9] U. W. Wydział Matematyki Informatyki i Mechaniki, „Sztuczna inteligencja/SI Moduł 13 - Uczenie się ze wzmocnieniem,” 2023, Dostęp: 2026-09-01.
- [10] V. Mnih i in., „Human-level control through deep reinforcement learning,” *Nature*, t. 518, s. 529–533, 2015, Dostęp: 2026-09-01. DOI: 10.1038/nature14236
- [11] V. Nair i G. E. Hinton, „Rectified Linear Units Improve Restricted Boltzmann Machines,” 2010, s. 807–814.
- [12] GeeksforGeeks, „Epsilon-Greedy Algorithm in Reinforcement Learning,” 2023, Dostęp: 2026-09-02.
- [13] R. S., „Introduction to Experience Replay for Off-Policy Deep Reinforcement Learning,” 2022, Dostęp: 2026-09-03.
- [14] K. Pykes, „Understanding the Bellman Equation in Reinforcement Learning,” 2024, Dostęp: 2026-09-02.
- [15] Zilliz, „How does the discount factor (gamma) affect RL training?,” 2025, Dostęp: 2026-09-02.

- [16] E. I. C. IT, „Jakie jest znaczenie współczynnika dyskonta (γ) w kontekście uczenia się przez wzmacnianie i jaki ma on wpływ na szkolenie i wydajność agenta DRL?,” 2024, Dostęp: 2026-09-01.
- [17] R. Alake, „Loss Functions in Machine Learning Explained,” 2026, Dostęp: 2026-09-03.
- [18] D. P. Kingma i J. Ba, *Adam: A Method for Stochastic Optimization*, <https://arxiv.org/abs/1412.6980>, Dostęp: 2026-09-01, 2015. DOI: 10.48550/arXiv.1412.6980
- [19] I. Goodfellow, Y. Bengio i A. Courville, *Deep Learning*. Cambridge, Massachusetts: MIT Press, 2016.
- [20] V. Mnih i in., „Asynchronous Methods for Deep Reinforcement Learning,” t. 48, 2016, s. 1928–1937. DOI: 10.48550/arXiv.1602.01783